# Task 1: Familiarize Yourself with the Dataset

## Import all the datasets

In [1]:
```
import pandas as pd

sessions_df = pd.read_csv("data/sessions.csv")
streamers_df = pd.read_csv("data/streamers.csv")
streams_df = pd.read_csv("data/streams.csv")
viewers_df = pd.read_csv("data/viewers.csv")
```

## Read the head of all the datasets to understand column format

In [2]:
```
sessions_df.head()
```

Out[2]:

| | session_id | viewer_id | streamer_id | stream_id | started_at | ended_at | duration_mins | chat_messages_s |
|---|---|---|---|---|---|---|---|---|
| **0** | SES000001 | VWR00938 | STR0055 | SM00860 | 2024-01-18 19:16:00 | 2024-01-18 20:28:00 | 72 | |
| **1** | SES000002 | VWR00804 | STR0034 | SM00542 | 2024-01-04 17:11:00 | 2024-01-04 19:41:00 | 150 | |
| **2** | SES000003 | VWR00704 | STR0008 | SM00124 | 2024-02-15 15:11:00 | 2024-02-15 16:53:00 | 102 | |
| **3** | SES000004 | VWR00926 | STR0024 | SM00373 | 2024-01-18 19:59:00 | 2024-01-18 20:56:00 | 57 | |
| **4** | SES000005 | VWR00883 | STR0008 | SM00120 | 2024-03-19 20:08:00 | 2024-03-19 20:22:00 | 14 | |

In [3]:
```
sessions_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 22825 entries, 0 to 22824
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   session_id          22825 non-null  str
 1   viewer_id           22825 non-null  str
 2   streamer_id         22765 non-null  str
 3   stream_id           22825 non-null  str
 4   started_at          22825 non-null  str
 5   ended_at            22825 non-null  str
 6   duration_mins       22825 non-null  int64
 7   chat_messages_sent  22825 non-null  int64
 8   bits_cheered        22825 non-null  int64
 9   followed_during     22825 non-null  bool
 10  subscribed_during   22825 non-null  bool
dtypes: bool(2), int64(3), str(6)
memory usage: 1.6 MB
```

In [4]:
```
streamers_df.head()
```

Out[4]:

| | streamer_id | streamer_name | category | language | partner_status | total_followers | avg_concurrent_viewe |
|---|---|---|---|---|---|---|---|
| **0** | STR0001 | xQc | Just Chatting | English | True | 11500000 | 13870 |
| **1** | STR0002 | Pokimane | Just Chatting | English | True | -25551 | 7166 |
| **2** | STR0003 | HasanAbi | Just Chatting | English | True | 2600000 | 3422 |
| **3** | STR0004 | Amouranth | Just Chatting | English | True | 6200000 | 6860 |
| **4** | STR0005 | Mizkif | Just Chatting | English | True | 2100000 | 677 |

In [5]:
```
streamers_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 76 entries, 0 to 75
Data columns (total 7 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   streamer_id            76 non-null     str
 1   streamer_name          76 non-null     str
 2   category               76 non-null     str
 3   language               76 non-null     str
 4   partner_status         76 non-null     bool
 5   total_followers        76 non-null     int64
 6   avg_concurrent_viewers  76 non-null     int64
dtypes: bool(1), int64(2), str(4)
memory usage: 3.8 KB
```

In [6]:
```
streams_df.head()
```

Out[6]:

| | stream_id | streamer_id | started_at | ended_at | stream_duration_hrs | category | peak_viewers | title_has_ |
|---|---|---|---|---|---|---|---|---|
| **0** | SM00001 | STR0001 | 2024-02-14 18:00:00 | 2024-02-14 21:00:00 | 3.0 | Just Chatting | 108989.0 | |
| **1** | SM00002 | STR0001 | 2024-01-28 18:00:00 | 2024-01-28 21:00:00 | 3.0 | Just Chatting | 51635.0 | |
| **2** | SM00003 | STR0001 | 2024-01-31 09:00:00 | 2024-01-31 16:00:00 | 7.0 | Just Chatting | 212127.0 | |
| **3** | SM00004 | STR0001 | 2024-03-03 14:00:00 | 2024-03-03 21:00:00 | 7.0 | Just Chatting | 186745.0 | |
| **4** | SM00005 | STR0001 | 2024-01-24 18:00:00 | 2024-01-24 21:00:00 | 3.0 | Just Chatting | 87723.0 | |

In [7]:
```
streams_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1199 entries, 0 to 1198
Data columns (total 9 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   stream_id            1199 non-null   str
 1   streamer_id          1199 non-null   str
 2   started_at           1199 non-null   str
 3   ended_at             1199 non-null   str
 4   stream_duration_hrs  1199 non-null   float64
 5   category             1149 non-null   str
 6   peak_viewers         1119 non-null   float64
 7   title_has_hype_word  1199 non-null   bool
 8   was_raid             1199 non-null   bool
dtypes: bool(2), float64(2), str(5)
memory usage: 68.0 KB
```

In [8]:
```
viewers_df.head()
```

Out[8]:

| | viewer_id | age_group | country | account_age_days | subscription_tier | preferred_category |
|---|---|---|---|---|---|---|
| **0** | VWR00001 | 18-24 | AR | 355 | free | IRL |
| **1** | VWR00002 | 25-34 | CA | 1670 | free | Creative |
| **2** | VWR00003 | 35-44 | US | 1108 | free | Just Chatting |
| **3** | VWR00004 | 45+ | NaN | 822 | tier1 | FPS |
| **4** | VWR00005 | 25-34 | DE | 2226 | free | Just Chatting |

In [9]:
```
viewers_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   viewer_id           1000 non-null   str
 1   age_group           1000 non-null   str
 2   country             920 non-null    str
 3   account_age_days    1000 non-null   int64
 4   subscription_tier   1000 non-null   str
 5   preferred_category  1000 non-null   str
dtypes: int64(1), str(5)
memory usage: 47.0 KB
```

Covert dates/times into pandas date/time objects to manipulate easier

In [10]:
```
# sessions.csv
sessions_df['started_at'] = pd.to_datetime(sessions_df['started_at'], format='%Y-%m-%d %H:%M:%S')
sessions_df['ended_at'] = pd.to_datetime(sessions_df['ended_at'], format='%Y-%m-%d %H:%M:%S')

# streams.csv
streams_df['started_at'] = pd.to_datetime(streams_df['started_at'], format='%Y-%m-%d %H:%M:%S')
streams_df['ended_at'] = pd.to_datetime(streams_df['ended_at'], format='%Y-%m-%d %H:%M:%S')
```

In [11]:
```
# check that it worked
sessions_start_date = sessions_df['started_at']
streams_start_date = streams_df['started_at']
print(sessions_start_date)
```

```
print()
print(streams_start_date)
```

```
0       2024-01-18 19:16:00
1       2024-01-04 17:11:00
2       2024-02-15 15:11:00
3       2024-01-18 19:59:00
4       2024-03-19 20:08:00
                ...
22820   2024-01-16 18:51:00
22821   2024-01-08 20:28:00
22822   2024-03-26 04:15:00
22823   2024-01-11 20:02:00
22824   2024-01-13 11:23:00
Name: started_at, Length: 22825, dtype: datetime64[us]

0      2024-02-14 18:00:00
1      2024-01-28 18:00:00
2      2024-01-31 09:00:00
3      2024-03-03 14:00:00
4      2024-01-24 18:00:00
               ...
1194   2024-01-31 19:00:00
1195   2024-03-20 19:00:00
1196   2024-01-01 14:00:00
1197   2024-01-02 20:00:00
1198   2024-02-19 08:00:00
Name: started_at, Length: 1199, dtype: datetime64[us]
```

## Task 1 Summary Report

I loaded the head and the general info of each dataset to quickly summarize whats in each set. When there was a date or time value, I converted it to a pandas datetime object.

### 1. sessions.csv = sessions_df

- total 11 columns
- range: 22825 entries, 0 to 22824
- dtypes: bool(2), datetime64us, int64(3), str(4)
- 'streamer_id' has 100 null entries

---

- *session_id, viewer_id, streamer_id, stream_id* = str
- *started_at, ended_at* = datetime64[us]
- *duration_mins, chat_messages_sent, bits_cheered* = int64
- *followed_during, subscribed_during* = bool

### 2. streamers.csv = streamers_df

- total 7 columns
- range: 76 entries, 0 to 75
- dtypes: bool(1), int64(2), str(4)
- 'language' has 8 null entries

---

- *streamer_id, streamer_name, category, language* = str
- *total_followers, avg_concurrent_viewers* = int64
- *partner_status* = bool

### 3. streams.csv = streams_df

- total 9 columns
- range: 1199 entries, 0 to 1198
- dtypes: bool(2), datetime64us, float64(2), str(3)
- 'category' has 50 null entries

---

- *category, streamer_id, stream_id* = str
- *started_at, ended_at* = datetime64[us]
- *peak_viewers, stream_duration_hrs* = float64
- *title_has_hype_word, was_raid* = bool

### 4. viewers.csv = viewers_df

- total 6 columns
- range: 1000 entries, 0 to 999
- dtypes: int64(1), str(5)
- 'country' has 80 null entries

---

- *age_group, viewer_id, country, subscription_tier, preferred_category* = str
- *account_age_days* = int64

#### Foreign Keys

sessions, streamers, & streams: *streamer_id*

viewers & sessions: *viewer_id*

sessions & streams: *stream_id*

# Task 2: Data Cleaning, Diagnostics, and Corrections

## Systemic Problems

### viewers.csv

- Missing values for 'country' (80)

Resolved by adding new country label 'unknown' to stay specific but not drop potentially important entries.

In [12]:
```
viewers_df_resolved = viewers_df.copy()

viewers_df_resolved['country'] = viewers_df_resolved['country'].fillna('unknown')

viewers_df_resolved.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   viewer_id           1000 non-null   str
 1   age_group           1000 non-null   str
 2   country             1000 non-null   str
 3   account_age_days    1000 non-null   int64
 4   subscription_tier   1000 non-null   str
 5   preferred_category  1000 non-null   str
dtypes: int64(1), str(5)
memory usage: 47.0 KB
```

### sessions.csv

- Missing values for 'streamer_id' (100)

Resolved by adding new value 'unknown' to stay specific but not drop potentially important entries.

In [13]:
```
sessions_df_resolved = sessions_df.copy()

sessions_df_resolved['streamer_id'] = sessions_df_resolved['streamer_id'].fillna('unknown')

sessions_df_resolved.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 22825 entries, 0 to 22824
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   session_id          22825 non-null  str
 1   viewer_id           22825 non-null  str
 2   streamer_id         22825 non-null  str
 3   stream_id           22825 non-null  str
 4   started_at          22825 non-null  datetime64[us]
 5   ended_at            22825 non-null  datetime64[us]
 6   duration_mins       22825 non-null  int64
 7   chat_messages_sent  22825 non-null  int64
 8   bits_cheered        22825 non-null  int64
 9   followed_during     22825 non-null  bool
 10  subscribed_during   22825 non-null  bool
dtypes: bool(2), datetime64[us](2), int64(3), str(4)
memory usage: 1.6 MB
```

- Some entries for 'started_at' and 'ended_at' are swapped
  - For instance, some streams end at an earlier date than when it started.

> Found by Tim K. on Canvas

For this, I decided to swap the values as they seemed to be an input glitch and not any other kind of error. Once swapped, the values make sense.

In [14]:
```
# create mask for sessions that end earlier than they start and swap them
swap_mask = sessions_df["started_at"] > sessions_df["ended_at"]

#check which ones are the broken entries
sessions_df.loc[swap_mask].head()
```

Out[14]:

| | session_id | viewer_id | streamer_id | stream_id | started_at | ended_at | duration_mins | chat_messag |
|---|---|---|---|---|---|---|---|---|
| **341** | SES000342 | VWR00878 | STR0010 | SM00147 | 2024-03-19 13:53:00 | 2024-03-19 12:15:00 | 98 | |
| **722** | SES000723 | VWR00859 | STR0020 | SM00311 | 2024-03-15 14:53:00 | 2024-03-15 11:57:00 | 176 | |
| **1011** | SES001012 | VWR00722 | STR0076 | SM01199 | 2024-02-19 08:36:00 | 2024-02-19 08:18:00 | 18 | |
| **1718** | SES001719 | VWR00916 | STR0073 | SM01160 | 2024-02-13 19:38:00 | 2024-02-13 19:36:00 | 2 | |
| **1897** | SES001898 | VWR00947 | STR0020 | SM00303 | 2024-01-01 11:18:00 | 2024-01-01 09:14:00 | 124 | |

In [15]:
```
#swap the values for ended_at and started_at
sessions_df_resolved.loc[swap_mask, ["started_at", "ended_at"]] = (
    sessions_df.loc[swap_mask, ["ended_at", "started_at"]].values
)

sessions_df_resolved.head(341)
```

Out[15]:

| | session_id | viewer_id | streamer_id | stream_id | started_at | ended_at | duration_mins | chat_messages |
|---|---|---|---|---|---|---|---|---|
| **0** | SES000001 | VWR00938 | STR0055 | SM00860 | 2024-01-18 19:16:00 | 2024-01-18 20:28:00 | 72 | |
| **1** | SES000002 | VWR00804 | STR0034 | SM00542 | 2024-01-04 17:11:00 | 2024-01-04 19:41:00 | 150 | |
| **2** | SES000003 | VWR00704 | STR0008 | SM00124 | 2024-02-15 15:11:00 | 2024-02-15 16:53:00 | 102 | |
| **3** | SES000004 | VWR00926 | STR0024 | SM00373 | 2024-01-18 19:59:00 | 2024-01-18 20:56:00 | 57 | |
| **4** | SES000005 | VWR00883 | STR0008 | SM00120 | 2024-03-19 20:08:00 | 2024-03-19 20:22:00 | 14 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **336** | SES000337 | VWR00886 | STR0032 | SM00511 | 2024-03-09 22:30:00 | 2024-03-09 23:08:00 | 38 | |
| **337** | SES000338 | VWR00624 | STR0070 | SM01105 | 2024-03-30 23:31:00 | 2024-03-30 23:33:00 | 2 | |
| **338** | SES000339 | VWR00680 | STR0074 | SM01172 | 2024-03-05 17:45:00 | 2024-03-05 19:08:00 | 83 | |
| **339** | SES000340 | VWR00435 | STR0038 | SM00611 | 2024-02-04 12:05:00 | 2024-02-04 12:19:00 | 14 | |
| **340** | SES000341 | VWR00670 | STR0051 | SM00792 | 2024-02-19 12:50:00 | 2024-02-19 13:10:00 | 20 | |

341 rows × 11 columns

- Session records reference non-existent streamers AND streams.

> Found by Tawshif Al Mehrab on Canvas

Resolved by removing these entries as they might be problematic for analysis.

I used this stackoverflow to help me combine both detections with a conditional: https://stackoverflow.com/questions/23199796/detect-and-exclude-outliers-in-a-pandas-dataframe

In [16]:
```
#rows where stream_id is missing from streams_df but present in sessions
#sessions_streamid_outliers = sessions_df[~sessions_df["stream_id"].isin(streams_df["stream_id"])

#rows where streamer_id is missing from streamers_df but present in sessions
#sessions_streamerid_outliers = sessions_df[~sessions_df["streamer_id"].isin(streamers_df["stream

#both conditions combined with either/or statement
#this contains all the rows with broken IDs in sessions_df
sessions_missingid_outliers = sessions_df[
    (~sessions_df["stream_id"].isin(streams_df["stream_id"])) |
    (~sessions_df["streamer_id"].isin(streamers_df["streamer_id"]))
]

#this only keeps the rows that aren't broken, we copy it to the new resolved df
sessions_df_resolved = sessions_df[
```

```
    (sessions_df["stream_id"].isin(streams_df["stream_id"])) &
    (sessions_df["streamer_id"].isin(streamers_df["streamer_id"]))
]
```

In [17]:
```
sessions_missingid_outliers.head()
```

Out[17]:

| | session_id | viewer_id | streamer_id | stream_id | started_at | ended_at | duration_mins | chat_messages |
|---|---|---|---|---|---|---|---|---|
| **126** | SES000127 | VWR00930 | STR0018 | SM99031 | 2024-03-10 16:24:00 | 2024-03-10 18:08:00 | 104 | |
| **475** | SES000476 | VWR00965 | STR0037 | SM99030 | 2024-01-02 00:21:00 | 2024-01-02 00:24:00 | 3 | |
| **489** | SES000490 | VWR00763 | NaN | SM01155 | 2024-03-05 10:25:00 | 2024-03-05 10:31:00 | 6 | |
| **519** | SES000520 | VWR00509 | NaN | SM01180 | 2024-01-02 20:26:00 | 2024-01-02 21:33:00 | 67 | |
| **847** | SES000848 | VWR00470 | NaN | SM00355 | 2024-02-05 20:25:00 | 2024-02-05 20:42:00 | 17 | |

In [18]:
```
sessions_df_resolved.info()
```

```
<class 'pandas.DataFrame'>
Index: 22730 entries, 0 to 22824
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   session_id          22730 non-null  str
 1   viewer_id           22730 non-null  str
 2   streamer_id         22730 non-null  str
 3   stream_id           22730 non-null  str
 4   started_at          22730 non-null  datetime64[us]
 5   ended_at            22730 non-null  datetime64[us]
 6   duration_mins       22730 non-null  int64
 7   chat_messages_sent  22730 non-null  int64
 8   bits_cheered        22730 non-null  int64
 9   followed_during     22730 non-null  bool
 10  subscribed_during   22730 non-null  bool
dtypes: bool(2), datetime64[us](2), int64(3), str(4)
memory usage: 1.8 MB
```

### streamers.csv

- Some follower counts are in the negative which is not possible. This is probably an input error.

Simply turned these into positive numbers by using pandas absolute function.

In [19]:
```
streamers_df["total_followers"] = streamers_df["total_followers"].abs()
```

- Missing values for 'language' (8)

Manually inputted in the language in the csv by looking up the streamers with that value missing

### streams.csv

- 'category' has 50 null entries

Resolved by adding new value 'unknown' to stay specific but not drop potentially important entries.

In [20]:
```
streams_df_resolved = streams_df.copy()

streams_df_resolved['category'] = streams_df_resolved['category'].fillna('unknown')

streams_df_resolved.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1199 entries, 0 to 1198
Data columns (total 9 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   stream_id            1199 non-null   str
 1   streamer_id          1199 non-null   str
 2   started_at           1199 non-null   datetime64[us]
 3   ended_at             1199 non-null   datetime64[us]
 4   stream_duration_hrs  1199 non-null   float64
 5   category             1199 non-null   str
 6   peak_viewers         1119 non-null   float64
 7   title_has_hype_word  1199 non-null   bool
 8   was_raid             1199 non-null   bool
dtypes: bool(2), datetime64[us](2), float64(2), str(3)
memory usage: 68.0 KB
```

- 'peak_viewers' has 80 null entries

Resolved by replacing it with the median instead of mean due to outliers being present in the form of viral streams.

In [21]:
```
streams_viewer_median = streams_df_resolved['peak_viewers'].median().round(2)

streams_df_resolved['peak_viewers'] = streams_df['peak_viewers'].fillna(streams_viewer_median)

print(f"Median used for replacement: {streams_viewer_median:.2f}")

# Check data has been fully resolved by looking at rows where peak viewers & category is missing
streams_df_resolved.head(25).tail(5)
```

```
Median used for replacement: 20224.00
```

Out[21]:

| | stream_id | streamer_id | started_at | ended_at | stream_duration_hrs | category | peak_viewers | title_has |
|---|---|---|---|---|---|---|---|---|
| **20** | SM00021 | STR0002 | 2024-03-23 22:00:00 | 2024-03-23 23:30:00 | 1.5 | unknown | 317.0 | |
| **21** | SM00022 | STR0002 | 2024-02-19 13:00:00 | 2024-02-19 18:00:00 | 5.0 | unknown | 20224.0 | |
| **22** | SM00023 | STR0002 | 2024-01-29 11:00:00 | 2024-01-29 13:30:00 | 2.5 | Just Chatting | 300.0 | |
| **23** | SM00024 | STR0002 | 2024-03-22 11:00:00 | 2024-03-22 18:00:00 | 7.0 | Just Chatting | 183.0 | |
| **24** | SM00025 | STR0002 | 2024-01-05 12:00:00 | 2024-01-05 20:00:00 | 8.0 | Just Chatting | 463.0 | |

## Heuristics

viewers.csv

- 'free', 'free trial', and 'none' are the same account type

Replaced with 'free'

In [22]:
```
account_combine = ["free", "free trial", "none"]

viewers_df["subscription_tier"] = viewers_df_resolved["subscription_tier"].replace(account_combir
```

- 'tier 4' and 'premium' as an account type does not actually exist. The maximum tier is 3.
- Some (5) rows contain 'TIER1' instead of 'tier1'

> Found by Johann Ferrera and Zoe Mann on Canvas

replaced with 'tier3'

'TIER1' replaced with 'tier1'

In [23]:
```
#replace tier4/preimum witn tier3
nonexistent_acc_types = ["tier4", "premium"]

viewers_df_resolved["subscription_tier"] = viewers_df["subscription_tier"].replace(nonexistent_ac

#replace TIER1 with tier1
viewers_df_resolved["subscription_tier"] = viewers_df_resolved["subscription_tier"].replace("TIE
```

- Inconsistent age data convention, for instance '18 to 24' instead of the usual '18-24'

> Additionally, some inputs use an en-dash instead of a hyphen. Found by Michael Mora & Atif Mahmud on Canvas.

Replace them in their strings directly, regex=False means it will treat those specific characters as literals not conditionals

In [24]:
```
# replace 'to' to - using original dataset first
viewers_df_resolved["age_group"] = viewers_df["age_group"].str.replace(" to ", "-", regex=False)

# replace endash to - in resolved one
viewers_df_resolved["age_group"] = viewers_df_resolved["age_group"].str.replace("–", "-", regex=F

#double check, in the .csv we can see entry 110 has '35 to 44'
#print to see if its changed to a hyphen
viewers_df_resolved.head(111)
```

Out[24]:

| | viewer_id | age_group | country | account_age_days | subscription_tier | preferred_category |
|---|---|---|---|---|---|---|
| **0** | VWR00001 | 18-24 | AR | 355 | free | IRL |
| **1** | VWR00002 | 25-34 | CA | 1670 | free | Creative |
| **2** | VWR00003 | 35-44 | US | 1108 | free | Just Chatting |
| **3** | VWR00004 | 45+ | unknown | 822 | tier1 | FPS |
| **4** | VWR00005 | 25-34 | DE | 2226 | free | Just Chatting |
| **...** | ... | ... | ... | ... | ... | ... |
| **106** | VWR00107 | 35-44 | US | 1802 | free | MOBA |
| **107** | VWR00108 | 13-17 | AU | 3437 | free | Just Chatting |
| **108** | VWR00109 | 35-44 | MX | 2251 | tier1 | FPS |
| **109** | VWR00110 | 25-34 | AU | 1519 | free | Sports |
| **110** | VWR00111 | 18-24 | ES | 1574 | free | Sports |

111 rows × 6 columns

### sessions.csv

Some streams have the same started_at and ended_at time/date

> Some bits and followers were gained during these streamers which is not possible

Decided to remove them as they are problematic entries with more than one problematic value, such as gaining bits or followers.

In [25]:
```
# only keep rows where started_at and ended_at are not the same
sessions_df_resolved = sessions_df_resolved[sessions_df_resolved["started_at"] != sessions_df_res
```

## Task 2 summary report

### Systemic Problems

viewers.csv

- **Missing Country Values (80 entries):** Applied `.fillna('unknown')` to retain rows with missing fields while still flagging them

sessions.csv

- **Missing Streamer IDs (100 entries):** Handled the same to the missing country values using `.fillna('unknown')`
- **Swapped Timestamps:** Fixed an input glitch where stream end times occurred before start times. Used a boolean comparison mask and extracted matrix properties with `.values` to swap values.
- **Relational Outliers:** Cleaned the dataset by applying AND ( `&` ) operator to retain only rows whose IDs matched valid IDs in `streams_df` and `streamers_df`.

streamers.csv

- **Negative Follower Counts:** Turned negative inputs into positive ones using `.abs()` which looked like an input glitch
- **Missing Language Values (8 entries):** Manually cross-referenced and updated the missing information directly.

streams.csv

- **Missing Category Entries (50 entries):** Applied `.fillna('unknown')` to missing entries without dropping the rows.

---

### Heuristic Adjustments

viewers.csv

- **Account Type Consolidation:** Combined multiple overlapping terms ('free', 'free trial', 'none') into a single `free` category using `.replace()`.
- **Invalid & Casing Anomalies:** Fixed non-existent tier classifications ('tier4', 'premium') by remapping them using `.replace()`. Fixed inconsistencies like `'TIER1'` by replacing them with the lowercase standard format
- **Age Group Format Alignment:** Fixed mismatched conventions (e.g., '18 to 24' and en-dashes) with `.str.replace()` to a standard hyphenated format (18-24).

sessions.csv

- **Identical Start/End Timestamps:** Removed zero-duration streaming logs containing impossible metrics (such as recorded bit/follower gains during non-existent runtimes) by using the inequality operator ( `!=` ).

# Task 3: Descriptive Statistics and Visualizations

## 1. Summary statistics to understand user characteristics & age groups

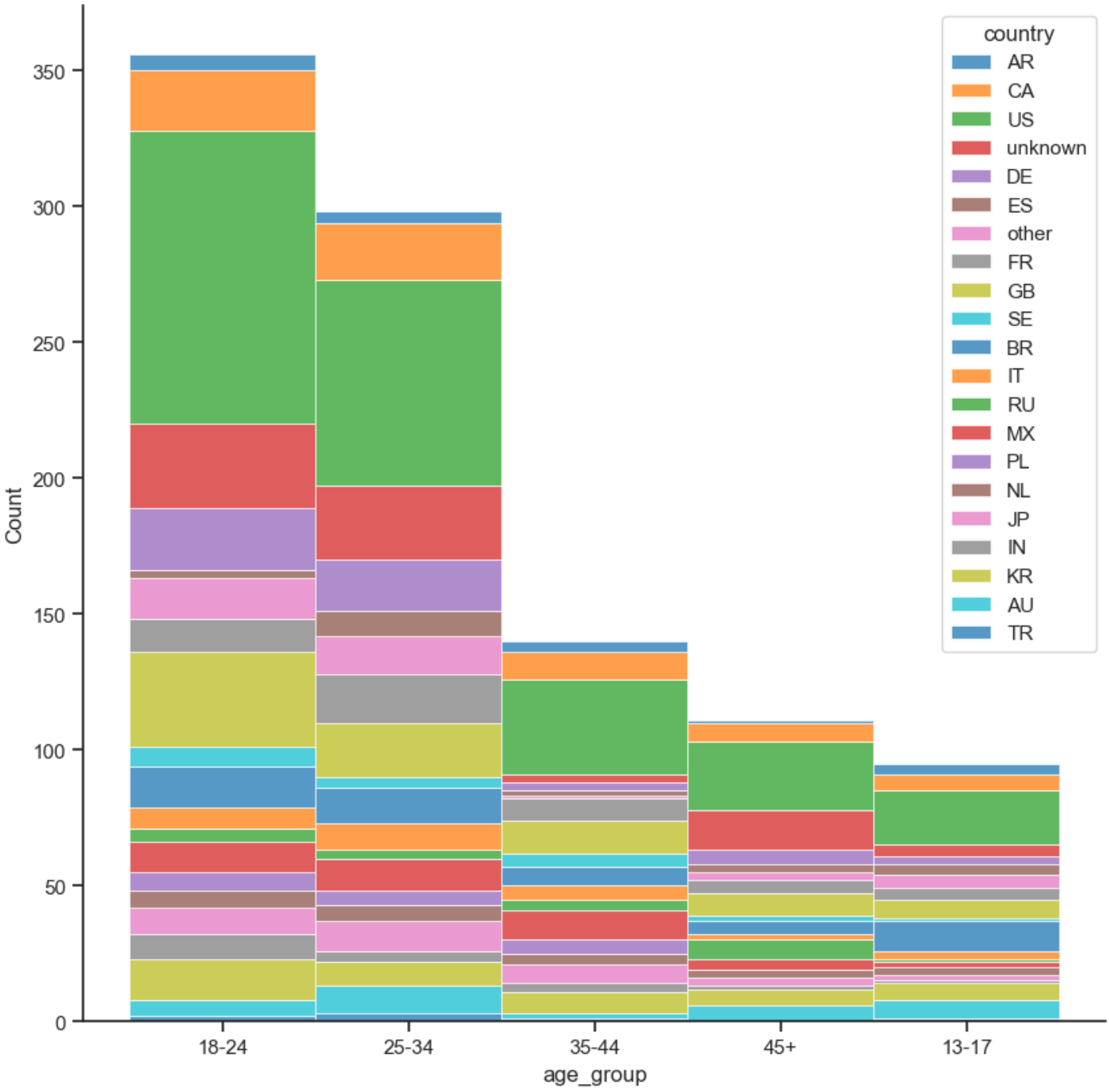
This includes tiers, age groups, and individual viewer interest

### Age Group of Viewers by Country

In [26]:
```
import seaborn as sns
import matplotlib as mpl
import matplotlib.pyplot as plt
import altair as alt
import numpy as np

# All the dataframes for reference:
# sessions_df_resolved, streams_df_resolved, viewers_df_resolved, streamers_df

sns.set_theme(style="ticks") # add ticks to the axes
```

```
f, ax = plt.subplots(figsize=(10, 10))
sns.despine(f)

sns.histplot(
    viewers_df_resolved,
    x="age_group", hue="country",
    multiple="stack", #put on top of eachother
    palette="tab10",
    edgecolor="1", #stroke 1=white
    linewidth=.5,
)
```

Out[26]: <Axes: xlabel='age_group', ylabel='Count'>




**Altair version of the graph using the documentation**

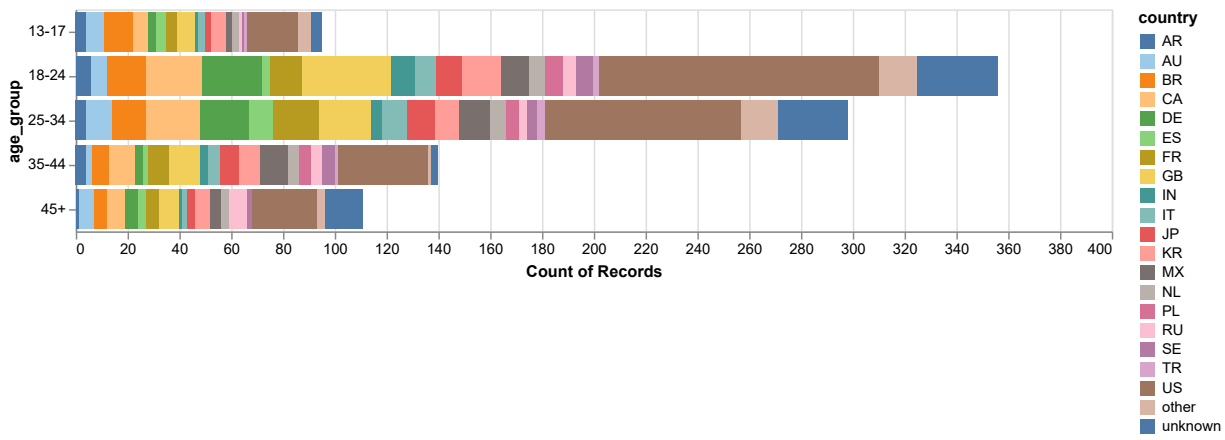
- https://altair-viz.github.io/altair-viz-v4/gallery/horizontal_stacked_bar_chart.html
- https://altair-viz.github.io/altair-viz-v4/user_guide/customization.html?highlight=color

I first tried with Seaborn since thats what we did in the lab, but I like Altair better because its similar to what

we did in vegalite

In [27]:
```
alt.Chart(viewers_df_resolved).mark_bar().encode(
    x='count():Q',
    y='age_group:N',
    color=alt.Color('country', scale=alt.Scale(scheme='tableau20')),
).properties(
    width=700,
    height=alt.Step(30)
)
```

Out[27]:



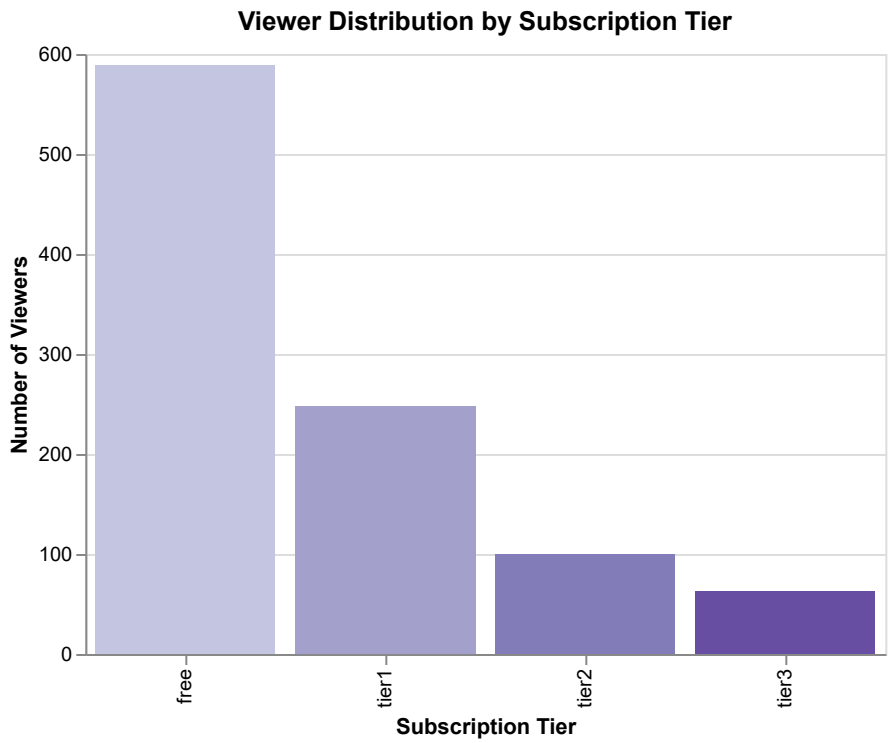

## Viewer Count by Subscription Tier

Used altair methods `x=alt.x()` for this instead of keywords & tooltips

- https://altair-viz.github.io/user_guide/encodings/index.html

In [28]:
```
alt.Chart(viewers_df_resolved).mark_bar(color="purple").encode(
    x=alt.X(
        "subscription_tier:N",
        title="Subscription Tier",
        sort=["free", "tier1", "tier2", "tier3"],
        ),
    y=alt.Y("count():Q", title="Number of Viewers"),
    color=alt.Color(
        "subscription_tier:N", scale=alt.Scale(scheme="purples"), legend=None
    ),
    tooltip=["subscription_tier", "count()"],
).properties(title="Viewer Distribution by Subscription Tier", width=400, height=300)
```

Out[28]:




## Average Watch Time Across Different Ages

Here, I decided to try and merge all the streaming data since I was successful before in order to speed up the data grouping process.

In [29]:
```
# merge sessions with their viewer ids to see how many watch per session
# inner merge means only keys from both are kept (just in case i didnt clean good)
session_viewers = pd.merge(
    sessions_df_resolved,
    viewers_df_resolved,
    on="viewer_id",
    how="inner",
)

# combine it AGAIN with stream activity using stream_id to get full logistics
# we can't actually use this cause its way too big
full_stream_activity = pd.merge(
    session_viewers, streams_df_resolved, on="stream_id", how="inner"
)
```

In [30]:
```
full_stream_activity.head(1)
```

Out[30]:

| | session_id | viewer_id | streamer_id_x | stream_id | started_at_x | ended_at_x | duration_mins | chat_message |
|---|---|---|---|---|---|---|---|---|
| **0** | SES000001 | VWR00938 | STR0055 | SM00860 | 2024-01-18 19:16:00 | 2024-01-18 20:28:00 | 72 | |

1 rows × 24 columns

Then I focused on grouping and visualizing the age groups by duration

In [31]:
```
# Group age groups by how long they watch streams on average
ages_avg_watchtime = (
    full_stream_activity.groupby("age_group")["duration_mins"]
    .mean()
    .reset_index()
```

```
)
```

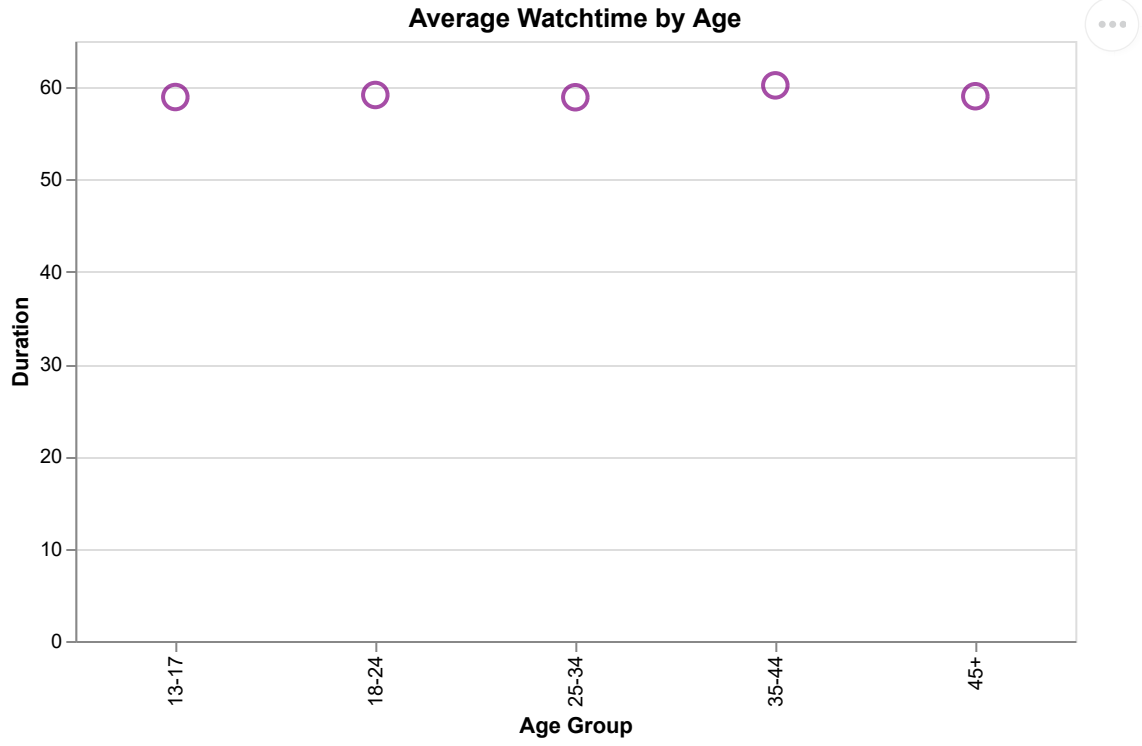
In [32]:
```
ages_avg_watchtime
```

Out[32]:

| | age_group | duration_mins |
|---|---|---|
| **0** | 13-17 | 58.929220 |
| **1** | 18-24 | 59.151183 |
| **2** | 25-34 | 58.906152 |
| **3** | 35-44 | 60.195145 |
| **4** | 45+ | 59.016429 |

In [33]:
```
alt.Chart(ages_avg_watchtime).mark_point(size=150, color = "purple").encode(
    y=alt.Y(
        "duration_mins:Q",
        title="Duration",
        axis=alt.Axis(format=",.0f"),
    ),
    x=alt.X("age_group:O", title="Age Group"),
    tooltip=[
        alt.Tooltip("age_group:N", title="Ages"),
        alt.Tooltip("duration_mins:Q", title="Average duration", format=",.0f"),
    ],
).properties(
    title="Average Watchtime by Age",
    width=500,
    height=300
)
```

Out[33]:

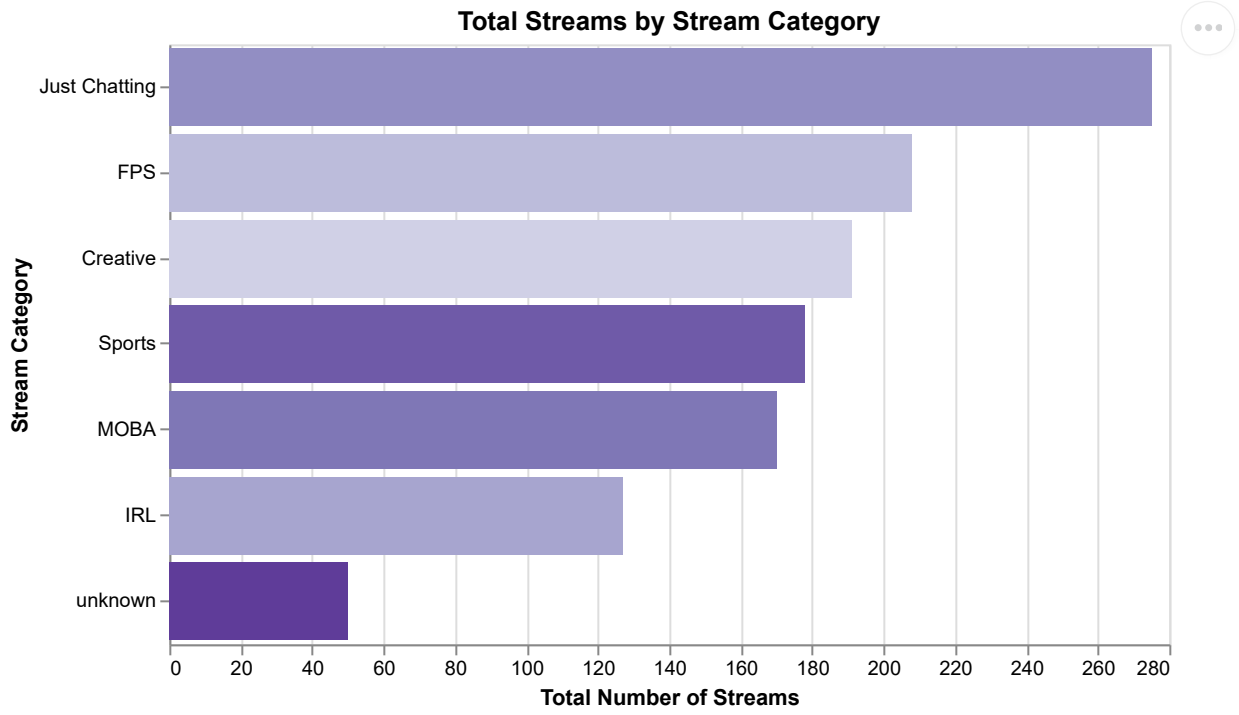



## 2. Summary statistics for distribution of streams, peak viewership, and data overall

## Total # of Streams by Stream Category

In [34]:
```
#nunique means only count unique stream sessions, no duplicates (for instance more than one viewe
total_stream_category = (
    full_stream_activity.groupby("category")["stream_id"]
    .nunique()
    .reset_index(name="stream_count")
)

alt.Chart(total_stream_category).mark_bar().encode(
    x=alt.X(
        "stream_count:Q",
        title="Total Number of Streams",
        axis=alt.Axis(format=",.0f"),
    ),
    y=alt.Y("category:N", title="Stream Category", sort="-x"),
    color=alt.Color(
        "category:N", scale=alt.Scale(scheme="purples"), legend=None
    ),
    tooltip=[
        alt.Tooltip("category:N", title="Category"),
        alt.Tooltip("stream_count:Q", title="Streams", format=",.0f"),
    ],
).properties(
    title="Total Streams by Stream Category",
    width=500,
    height=300,
)
```

Out[34]:




In [35]:
```
total_stream_category
```

Out[35]:

| | category | stream_count |
|---|---|---|
| **0** | Creative | 191 |
| **1** | FPS | 208 |
| **2** | IRL | 127 |
| **3** | Just Chatting | 275 |
| **4** | MOBA | 170 |
| **5** | Sports | 178 |
| **6** | unknown | 50 |

### Total Watch Time (Mins) of Viewers by Preferred Category

Used viewer_id to link duration_mins to viewer_id and then used that to display time for each preferred_category

- https://www.w3schools.com/python/pandas/ref_df_merge.asp

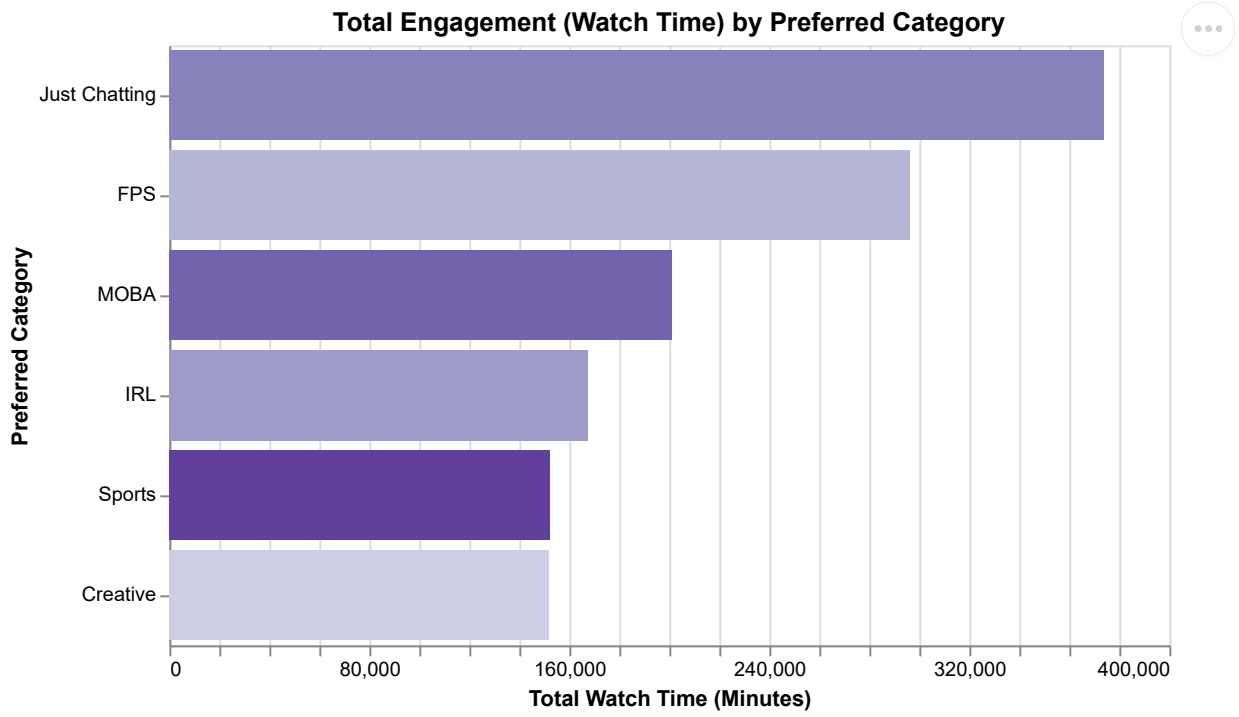
For reference, this was the code I wrote before creating `full_stream_activity` where I learned to merge and group data. I moved it to this section as it fit the instructions better.

In [36]:
```
#create new dataframe column for viewer watchtime by linking viewer_id in viewers_df to duration_
viewer_watchtime = (
    sessions_df_resolved.groupby("viewer_id")["duration_mins"].sum().reset_index()
)

#left joining: preserve key order & keeps all keys from the left DataFrame (which is viewers)
merged_viewers = pd.merge(viewers_df_resolved, viewer_watchtime, on="viewer_id", how="left")

# if viewer id is missing or doesnt match, fill with 0 mins (just in case)
merged_viewers["duration_mins"] = merged_viewers["duration_mins"].fillna(0)

# group by category
category_summary = (
    merged_viewers.groupby("preferred_category")["duration_mins"]
    .sum()
    .reset_index()
)
```

In [37]:
```
alt.Chart(category_summary).mark_bar().encode(
    x=alt.X(
        "duration_mins:Q",
        title="Total Watch Time (Minutes)",
        axis=alt.Axis(format=",.0f"),
    ),
    y=alt.Y("preferred_category:N", title="Preferred Category", sort="-x"),
    color=alt.Color(
        "preferred_category:N", scale=alt.Scale(scheme="purples"), legend=None
    ),
    tooltip=[
        alt.Tooltip("preferred_category:N", title="Category"),
        alt.Tooltip("duration_mins:Q", title="Total Minutes", format=",.0f"),
    ],
    ).properties(
        title="Total Engagement (Watch Time) by Preferred Category",
```

```
        width=500,
        height=300,)
```

Out[37]:




## Peak viewership for each month

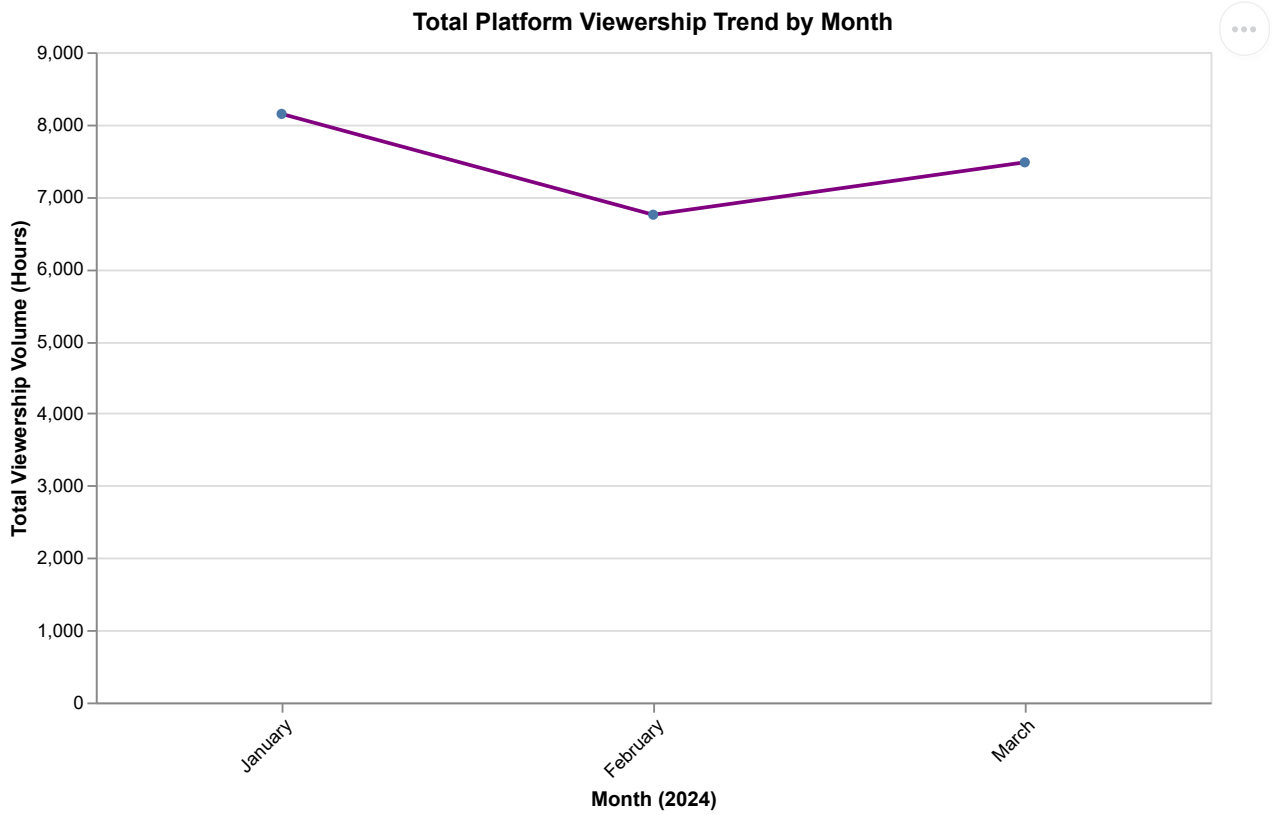
- I used this stack overflow to extract the month names from pandas datetime object
  - https://stackoverflow.com/questions/57033657/how-to-extract-month-name-and-year-from-date-column-of-dataframe

In [38]:

```
# Extract month name from our converted pandas date object
sessions_df_resolved['month'] = sessions_df_resolved['started_at'].dt.strftime('%B')

monthly_viewership = (sessions_df_resolved.groupby("month").agg(
    total_sessions=("session_id","count",),  # distinct watch sessions
    total_watch_hours=("duration_mins", lambda x: x.sum() / 60,),  # Summing total watch hours
).reset_index())

month_order = [
    "January",
    "February",
    "March",
    "April",
    "May",
    "June",
    "July",
    "August",
    "September",
    "October",
    "November",
    "December",
]

alt.Chart(monthly_viewership).mark_line(point=True, color="purple").encode(
    x=alt.X(
        "month:N",
        title="Month (2024)",
        sort=month_order,  # Enforces correct calendar sequence
        axis=alt.Axis(labelAngle=-45),
```

```
    ),
    y=alt.Y(
        "total_watch_hours:Q",
        title="Total Viewership Volume (Hours)",
        axis=alt.Axis(format=",.0f"),
    ),
    tooltip=[
        alt.Tooltip("month:N", title="Month"),
        alt.Tooltip("total_watch_hours:Q", title="Total Watch Hours", format=",.1f"),
        alt.Tooltip("total_sessions:Q", title="Total Sessions", format=","),
        ],
        ).properties(
            title="Total Platform Viewership Trend by Month",
            width=600,
            height=350)
```

Out[38]:



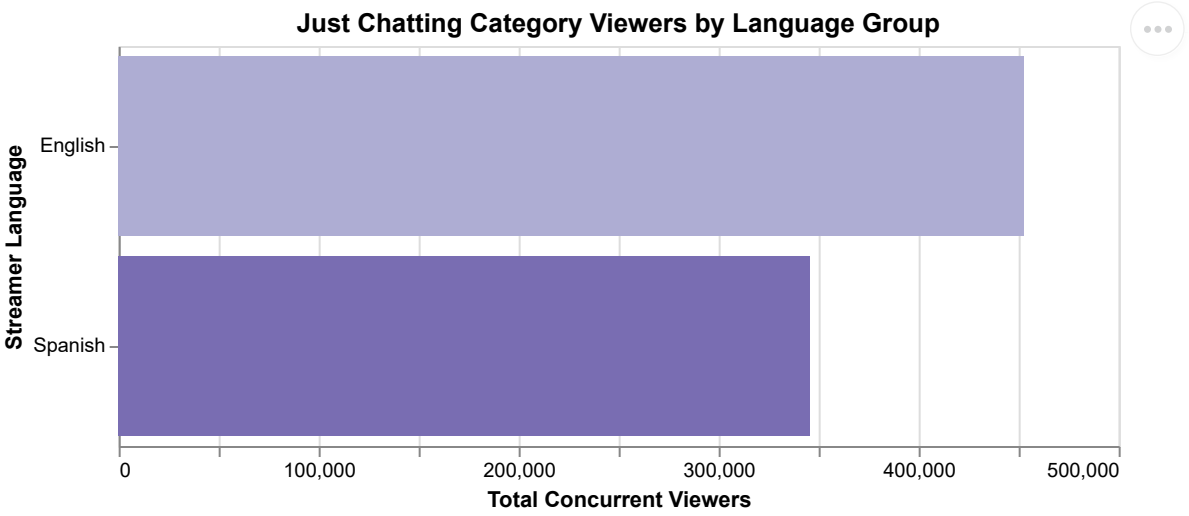

## 3. Show category-specific information and compare streamers based on categories

### "Just Chatting" Category by Language of Viewers

In [39]:

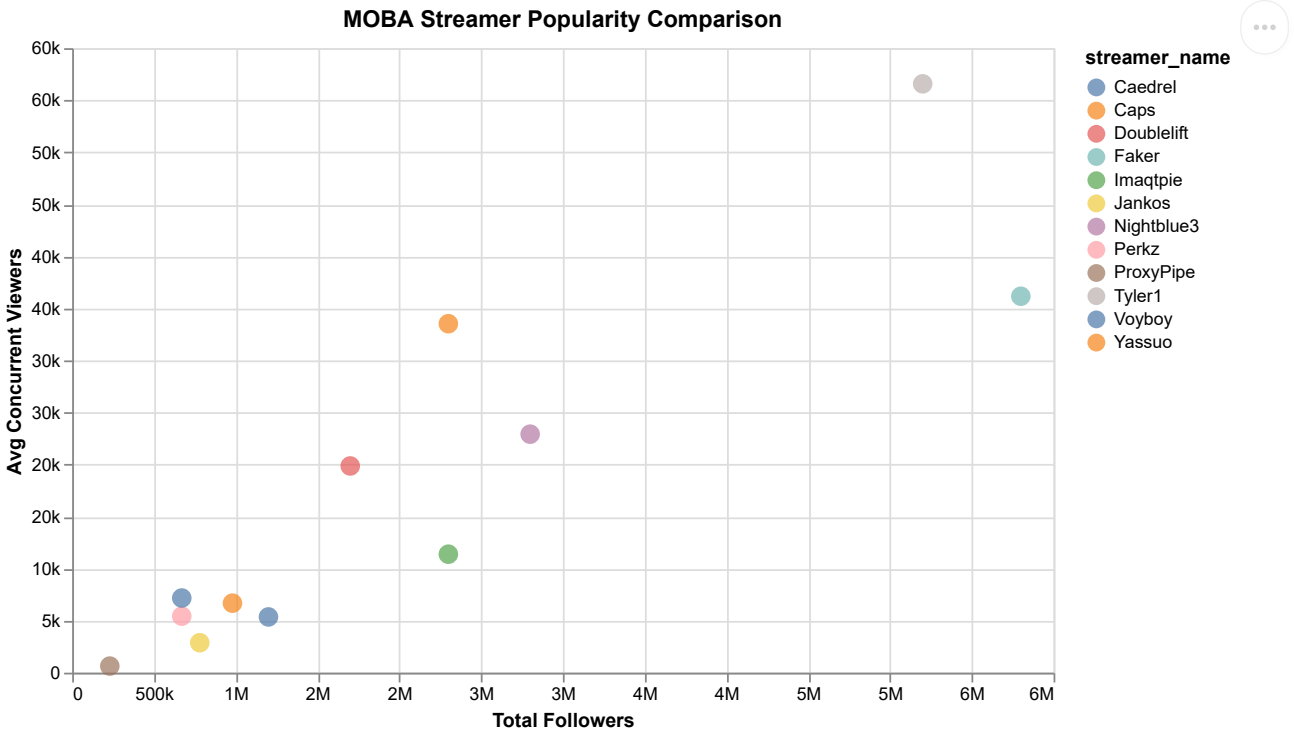
```
just_chatting_streamers = streamers_df[streamers_df["category"] == "Just Chatting"]

alt.Chart(just_chatting_streamers).mark_bar().encode(
    x=alt.X(
        "sum(avg_concurrent_viewers):Q",
        title="Total Concurrent Viewers",
        axis=alt.Axis(format=",.0f"),
    ),
    y=alt.Y("language:N", title="Streamer Language", sort="-x"),
    color=alt.Color(
        "language:N", scale=alt.Scale(scheme="purples"), legend=None
    ),
    tooltip=[
        alt.Tooltip("language:N", title="Language"),
```

```
        alt.Tooltip(
            "sum(avg_concurrent_viewers):Q",
            title="Total Viewers",
            format=",.0f",
        ),
    ],
).properties(
    title="Just Chatting Category Viewers by Language Group",
    width=500,
    height=200,
).display()
```




## Popularity Comparison Among MOBA Streamers

In [40]:
```
moba_data = streamers_df[streamers_df["category"] == "MOBA"]

alt.Chart(moba_data).mark_point(size=120, filled=True).encode(
    x=alt.X(
        "total_followers:Q",
        title="Total Followers",
        axis=alt.Axis(format=",.0s"),
    ),
    y=alt.Y(
        "avg_concurrent_viewers:Q",
        title="Avg Concurrent Viewers",
        axis=alt.Axis(format=",.0s"),
    ),
    color=alt.Color("streamer_name:N"),
    tooltip=[
        alt.Tooltip("streamer_name:N", title="Streamer"),
        alt.Tooltip("total_followers:Q", title="Followers", format=","),
        alt.Tooltip(
            "avg_concurrent_viewers:Q", title="Avg Viewers", format=","
        ),
    ],
).properties(
    title="MOBA Streamer Popularity Comparison",
    width=550,
    height=350,
).display()
```

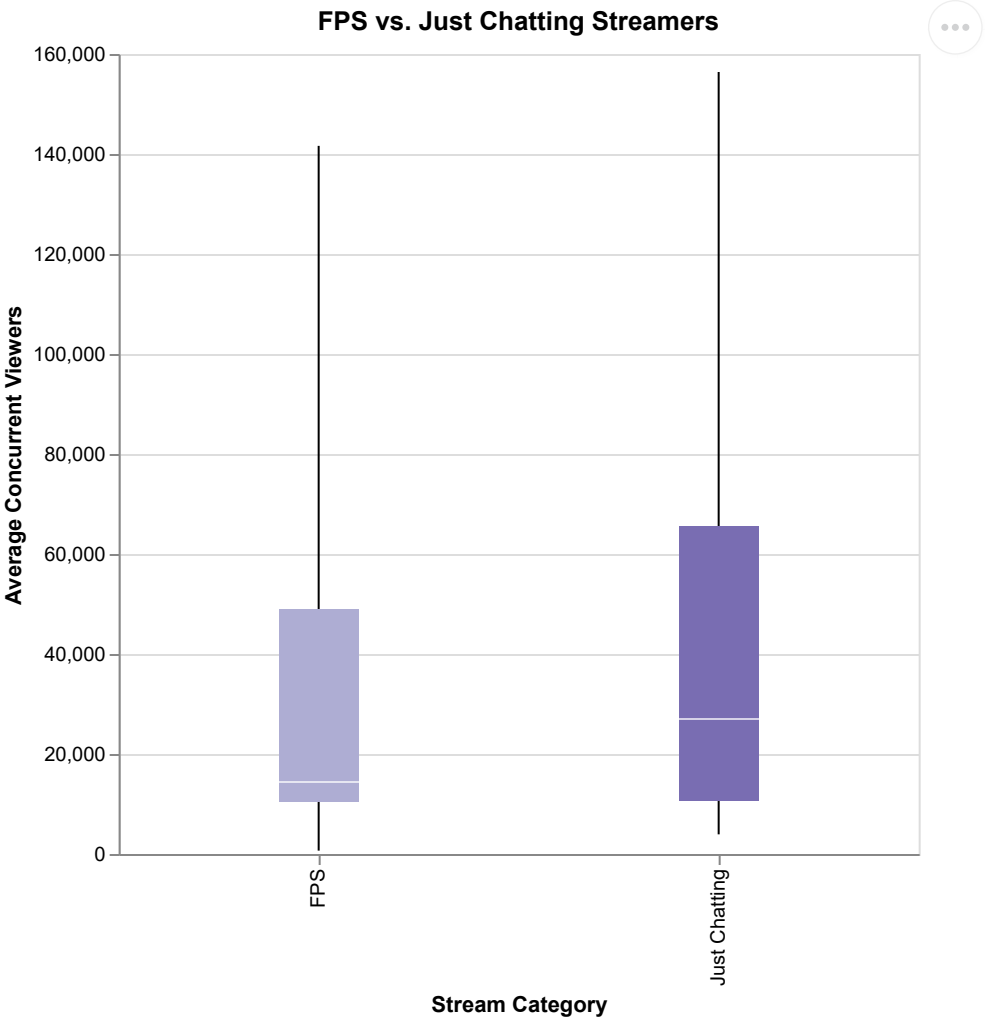



## Popularity of 'FPS' vs 'Just Chatting' streamers

Using average concurrent viewers, we can caluclate the mean/median/max of each stream category and then boxplot them to see where the extremes can reach vs. average

- Altair automatically calculates this for us in the tooltip

In [41]:
```
fps_vs_jc = streamers_df[streamers_df["category"].isin(["FPS", "Just Chatting"])]

alt.Chart(fps_vs_jc).mark_boxplot(extent="min-max", size=40).encode(
    x=alt.X("category:N", title="Stream Category"),
    y=alt.Y(
        "avg_concurrent_viewers:Q",
        title="Average Concurrent Viewers",
        axis=alt.Axis(format=",.0f"),
    ),
    color=alt.Color(
        "category:N",
        scale=alt.Scale(domain=["FPS", "Just Chatting"], scheme="purples"),
        legend=None,
    )
).properties(
    title="FPS vs. Just Chatting Streamers",
    width=400,
    height=400,
).display()
```




## Task 3 Summary report

### 1. User Characteristics & Age Demographics

- **Demographics by Country:** display the distribution of viewer age groups across different countries.
- **Subscription Tiers:** Isolated viewer counts by grouping categorical tiers using `sort=` to distinguish the free accounts through paid tiers ( `free` -> `tier1` -> `tier2` -> `tier3` ).
- **Average Watch Time by Age:** shared relational columns using an inner `.merge()` on `viewer_id` and `stream_id` . Grouped categories with `.groupby()` and calculated mean with `.mean()`

### 2. Stream Distribution & Overall Platform Metrics

- **Stream Category Sizing:** using `.groupby()` on categories combined with `.nunique()` to count unique instances of `stream_id`
- **Preferred Category Engagement:** Got total viewer engagement by calculating `.sum()` of total watch time per viewer. Merged them back before mapping the totals to a horizontal bar chart.
- **Chronological Trends:** Used `.dt.strftime('%B')` to isolate month names. I divided overall minutes by 60 to convert them into a hours scale using `.agg()`

### 3. Category-Specific Comparisons & Streamer Analytics

- **Just Chatting Languages:** Filtered streamers_df.csv to content creators under the `Just Chatting`

genre. Divided viewership metrics by language.

- **MOBA Peer Performance:** Extracted streamer popularity metrics within the `MOBA` category. Used a plot map to easily highlight top-performing creator outliers.
- **FPS vs. Just Chatting Benchmarks:** Used the method `.isin()` to filter down and compare the dataset's two largest genres.

# Task 4: Dealing with Outliers and Missing Values

I had some help with this section with this article by Hasan Ersan

- https://hersanyagci.medium.com/detecting-and-handling-outliers-with-pandas-7adbfcd5cad8

## Missing Values / Outliers in the original data

Analysis (see **Task 1**):

- sessions.csv = `streamer_id` has 100 null entries
- streamers.csv = `language` has 8 null entries
- streams.csv = `category` has 50 null entries
- viewers.csv = `country` has 80 null entries

I already dealt with these problems in **Task 2** so I'll just reiterate what I've done to explain my two strategies

### viewers.csv

- Missing values for 'country' (80)

**Strategy 1:** Resolved by adding new country label 'unknown' to stay specific but not drop potentially important entries.

In [42]:
```
viewers_df_resolved = viewers_df.copy()

viewers_df_resolved['country'] = viewers_df_resolved['country'].fillna('unknown')

viewers_df_resolved.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   viewer_id           1000 non-null   str
 1   age_group           1000 non-null   str
 2   country             1000 non-null   str
 3   account_age_days    1000 non-null   int64
 4   subscription_tier   1000 non-null   str
 5   preferred_category  1000 non-null   str
dtypes: int64(1), str(5)
memory usage: 47.0 KB
```

### streams.csv

- 'peak_viewers' has 80 null entries

**Strategy 2:** Resolved by replacing it with the median instead of mean due to outliers being present in the form of viral streams.

In [517…
```
# Please see code in Task 2 I don't want to accidentally run it twice
```

## Missing Values / Outliers in the derived data

First I use this function we defined in Lab02 to find outliers in the data using the +/- std

In [43]:
```
def detect_outliers(df, column):
    mean = df[column].mean() # the mean = mean of that specific data column
    std = df[column].std() # standard deviation
    lower_bound = mean - 2 * std # mean -2 std
    higher_bound = mean + 2 * std # mean +2 std
    return df[(df[column] < lower_bound) | (df[column] > higher_bound)]
```

### streams_df_resolved

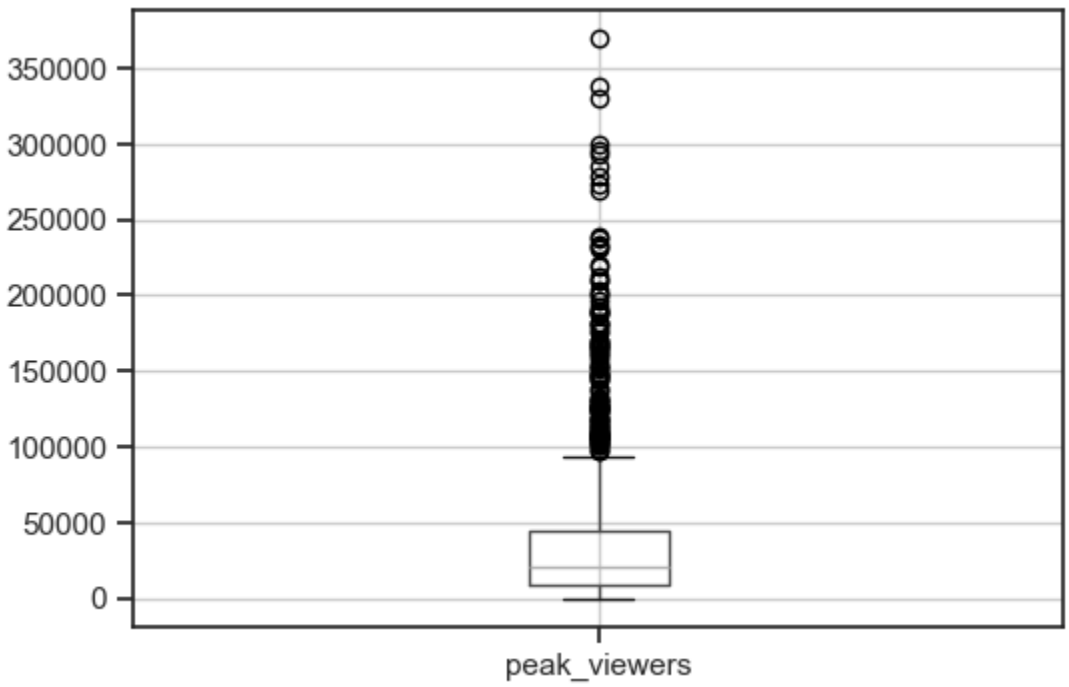
- there are around 61 extremely high preforming streams that don't really reflect the average peak twitch viewership count on a session

**Strategy 1:** Resolved by removing the outliers entirely

In [44]:
```
# First analyze the streams data using the function

streams_outliers = detect_outliers(streams_df_resolved, "peak_viewers")
streams_outliers.info()
print()
print(streams_outliers['peak_viewers'])
```

```
<class 'pandas.DataFrame'>
Index: 64 entries, 2 to 979
Data columns (total 9 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   stream_id            64 non-null     str
 1   streamer_id          64 non-null     str
 2   started_at           64 non-null     datetime64[us]
 3   ended_at             64 non-null     datetime64[us]
 4   stream_duration_hrs  64 non-null     float64
 5   category             64 non-null     str
 6   peak_viewers         64 non-null     float64
 7   title_has_hype_word  64 non-null     bool
 8   was_raid             64 non-null     bool
dtypes: bool(2), datetime64[us](2), float64(2), str(3)
memory usage: 4.1 KB

2      212127.0
3      186745.0
8      161545.0
11     148097.0
132    165095.0
         ...
969    296232.0
970    299413.0
973    210085.0
976    338648.0
979    330664.0
Name: peak_viewers, Length: 64, dtype: float64
```

In [45]:
```
# Visualize it with boxplot just to see if its accurate

streams_df_resolved.boxplot(column='peak_viewers', figsize=(6,4))
plt.show()
```




This is where I remove these extremely well-preforming streams using their indexes

In [46]:
```
streams_outliers_removed = streams_df_resolved.drop(streams_outliers.index)
print(streams_outliers_removed)
```

```
     stream_id streamer_id          started_at            ended_at  \
0      SM00001     STR0001 2024-02-14 18:00:00 2024-02-14 21:00:00
1      SM00002     STR0001 2024-01-28 18:00:00 2024-01-28 21:00:00
4      SM00005     STR0001 2024-01-24 18:00:00 2024-01-24 21:00:00
5      SM00006     STR0001 2024-01-09 14:00:00 2024-01-09 17:00:00
6      SM00007     STR0001 2024-03-05 12:00:00 2024-03-05 16:00:00
...        ...         ...                 ...                 ...
1194   SM01195     STR0076 2024-01-31 19:00:00 2024-01-31 23:00:00
1195   SM01196     STR0076 2024-03-20 19:00:00 2024-03-20 21:30:00
1196   SM01197     STR0076 2024-01-01 14:00:00 2024-01-01 21:00:00
1197   SM01198     STR0076 2024-01-02 20:00:00 2024-01-02 23:00:00
1198   SM01199     STR0076 2024-02-19 08:00:00 2024-02-19 10:00:00

      stream_duration_hrs       category  peak_viewers  title_has_hype_word  \
0                     3.0  Just Chatting      108989.0                False
1                     3.0  Just Chatting       51635.0                 True
4                     3.0  Just Chatting       87723.0                False
5                     3.0  Just Chatting       74062.0                 True
6                    48.0  Just Chatting       20224.0                 True
...                   ...            ...           ...                  ...
1194                  4.0            FPS        1421.0                False
1195                  2.5            FPS        2032.0                 True
1196                  7.0            FPS        1967.0                False
1197                  3.0            FPS        2041.0                False
1198                  2.0            FPS         771.0                False

      was_raid
0         True
1        False
4         True
5        False
6        False
...        ...
1194     False
1195     False
1196     False
1197     False
1198     False

[1135 rows x 9 columns]
```

In [47]:
```
print(f"Original dataset size: {len(streams_df_resolved)}")
print(f"After removing outliers: {len(streams_outliers_removed)}")
```

```
Original dataset size: 1199
After removing outliers: 1135
```

In [48]:
```
streams_outliers_new = detect_outliers(streams_outliers_removed, "peak_viewers")
streams_outliers_new.info()
print()
print(streams_outliers_new['peak_viewers'])
```

```
<class 'pandas.DataFrame'>
Index: 76 entries, 0 to 1131
Data columns (total 9 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   stream_id            76 non-null     str
 1   streamer_id          76 non-null     str
 2   started_at           76 non-null     datetime64[us]
 3   ended_at             76 non-null     datetime64[us]
 4   stream_duration_hrs  76 non-null     float64
 5   category             76 non-null     str
 6   peak_viewers         76 non-null     float64
 7   title_has_hype_word  76 non-null     bool
 8   was_raid             76 non-null     bool
dtypes: bool(2), datetime64[us](2), float64(2), str(3)
memory usage: 4.9 KB

0       108989.0
4        87723.0
7       118974.0
9       131075.0
57      107512.0
          ...
1114     92752.0
1118    102761.0
1123    107908.0
1128     91237.0
1131     87681.0
Name: peak_viewers, Length: 76, dtype: float64
```

By removing the outliers, the standard deviation gets reduced but also unfortunately permanetly deletes those viral streams with high amount of peak viewers. Now peak viewers never exceed 200000.

### streamers_df

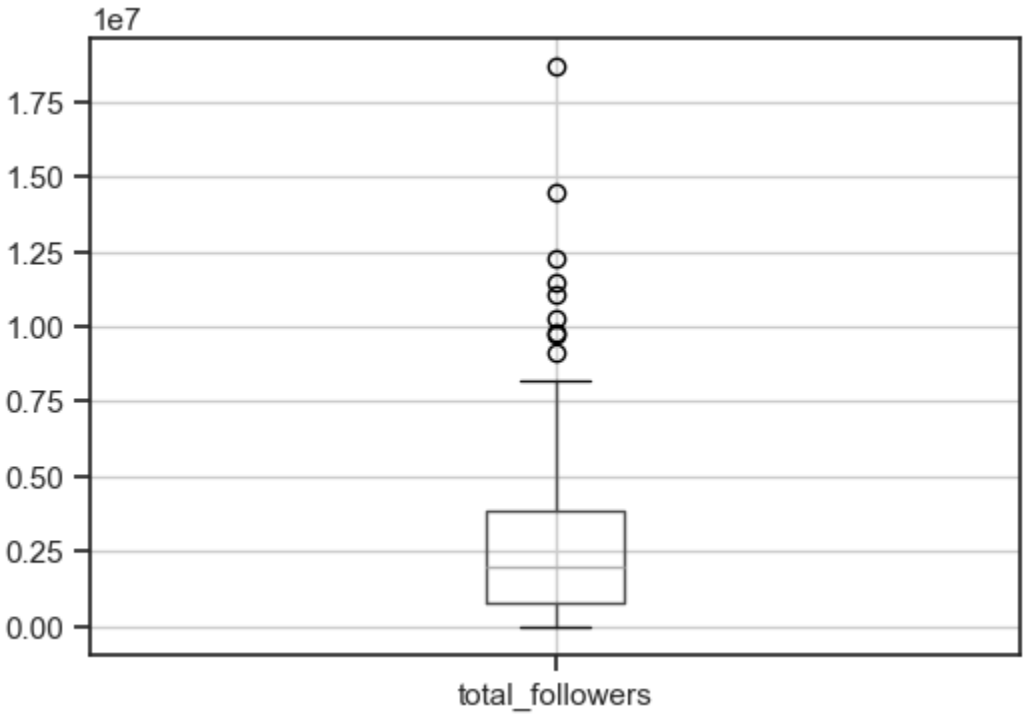
- there are around 5 streamers with extremely high total follower counts > 11100000 followers

**Strategy 2:** Normalize the follower count using log transformation to understand ratios/magnitude better

In [49]:
```python
# First analyze the streamers data using the outlier function
streamer_outliers = detect_outliers(streamers_df, "total_followers")
streamer_outliers.info()
print()
print(streamer_outliers['total_followers'])
```

```
<class 'pandas.DataFrame'>
Index: 5 entries, 0 to 61
Data columns (total 7 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   streamer_id            5 non-null      str
 1   streamer_name          5 non-null      str
 2   category               5 non-null      str
 3   language               5 non-null      str
 4   partner_status         5 non-null      bool
 5   total_followers        5 non-null      int64
 6   avg_concurrent_viewers  5 non-null      int64
dtypes: bool(1), int64(2), str(4)
memory usage: 285.0 bytes

0     11500000
8     14500000
18    11100000
60    12300000
61    18700000
Name: total_followers, dtype: int64
```

In [50]:
```
streamers_df.boxplot(column='total_followers', figsize=(6,4))
plt.show()
```



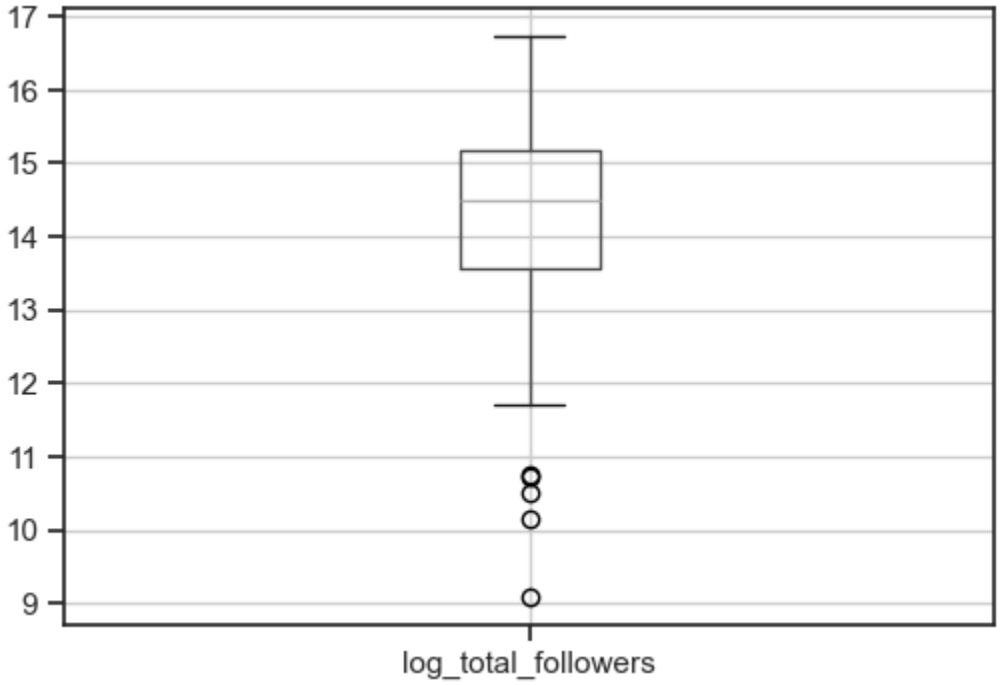

Apply the log transformations here and then check the boxplot again

In [51]:
```
streamers_log = np.log1p(streamers_df["total_followers"])
streamers_log.head(10)
```

Out[51]:
```
0    16.257858
1    10.148471
2    14.771022
3    15.640060
4    14.557448
5    13.764218
6    10.717325
7    10.757158
8    16.489659
9    16.087637
Name: total_followers, dtype: float64
```

In [52]:
```
#create new column for log transformations in dataset
streamers_df["log_total_followers"] = np.log1p(streamers_df["total_followers"])
streamers_df.boxplot(column="log_total_followers", figsize=(6, 4))
```

Out[52]:
```
<Axes: >
```




In [53]:
```
# Check new outliers in the logged data
streamer_log_outliers = detect_outliers(streamers_df, "log_total_followers")
streamer_log_outliers.info()
print()
print(streamer_log_outliers['log_total_followers'])
```

```
<class 'pandas.DataFrame'>
Index: 5 entries, 1 to 24
Data columns (total 8 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   streamer_id             5 non-null      str
 1   streamer_name           5 non-null      str
 2   category                5 non-null      str
 3   language                5 non-null      str
 4   partner_status          5 non-null      bool
 5   total_followers         5 non-null      int64
 6   avg_concurrent_viewers  5 non-null      int64
 7   log_total_followers     5 non-null      float64
dtypes: bool(1), float64(1), int64(2), str(4)
memory usage: 325.0 bytes

1     10.148471
6     10.717325
7     10.757158
12     9.094144
24    10.495045
Name: log_total_followers, dtype: float64
```

Instead of removing streamers with large followings, we just rescale the distrubition of total_followers. This reflects magnitude/scale rather than literal count.

## Task 4 Summary report

Please see Task 2 to see a summary of the entire cleaning and other imputation processes

Handling Missing Values in the Original Data

- **Viewer Country Fix (Strategy 1 - Placeholder):** The original user profile dataset had 80 blanks in the country column. Dropping these records risked losing valuable engagement metrics from other columns. This was resolved by applying `.fillna('unknown')` to insert a clean placeholder, preserving all 1,000 original profile entries.

- **Stream Peak Viewers Fix (Strategy 2 - Median Imputation):** The broadcast history dataset contained 80 missing entries for concurrent viewership. Because the platform contains massive viral stream spikes, using a standard average would have pulled values upward unfairly. This was resolved by substituting the blanks with the statistical median value of the category instead.

Managing Outliers in Derived Data

**Strategy 1: Removing Extreme Outliers (Peak Viewership)**

- **Process:** Running a standard deviation-based helper function revealed 64 hyper-viral streaming sessions where peak metrics sat multiple standard deviations above normal platform benchmarks. A boxplot visual confirmed these extreme values stretched heavily toward the top end. The records were extracted and physically discarded from the final working dataset using `.drop()`.

- **Result:** This dropped the total broadcast row count from 1,199 down to 1,135 streams. While it drastically lowered the data's standard deviation and capped the maximum viewer scale at a predictable 200,000 baseline, it permanently erased the historical footprint of major viral events from subsequent calculations.

**Strategy 2: Log Transformation (Total Follower Counts)**

- **Process:** The initial data scan isolated 5 massive platform celebrities with exceptional follower milestones exceeding 11 million users, creating a heavily skewed boxplot. Instead of deleting these highly important creators, a mathematical log transformation was introduced using `np.log1p()`. This new calculation was stored inside a distinct DataFrame column.
- **Result:** Rather than losing data points, this process squeezed the massive, multi-million follower gaps into a symmetrical range between roughly 9 and 16. Re-running the boxplot tool over this transformed scale confirmed that the distribution became normalized, switching the focal perspective to percentage-based magnitude and scale rather than raw numbers."""

# Task 5: Correlation and Behavioral Patterns

## 1. Compute pairwise correlations between user features and the number of check-ins

I'm not too sure what this means. I assume it means correlate user features like tier and the activity of users like watching streams/sessions, giving bits, and sending chat messages.

In [54]:
```
# count number of sessions and sum up the activity of the user in that id
user_activity = (sessions_df_resolved.groupby("viewer_id").agg(
        sessions=("session_id", "count"),
        total_watch_time=("duration_mins", "sum"),
        chat_messages=("chat_messages_sent", "sum"),
        bits_cheered=("bits_cheered", "sum"),
    ).reset_index()
)

# merge with the user 'features' like tier, country, & age group
# inner only keeps the entry if the viewer id is in both viewers.csv and our user_actitivty
features_df = pd.merge(
    viewers_df_resolved, user_activity, on="viewer_id", how="inner"
)

#preview & double check theres no null columns
features_df.head()
features_df.info()

#select these columns only to put into a new correlation dataframe
correlation_cols = [
    "account_age_days",
    "sessions",
    "total_watch_time",
    "chat_messages",
    "bits_cheered",
]

correlation_data = features_df[correlation_cols]
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   viewer_id           1000 non-null   str
 1   age_group           1000 non-null   str
 2   country             1000 non-null   str
 3   account_age_days    1000 non-null   int64
 4   subscription_tier   1000 non-null   str
 5   preferred_category  1000 non-null   str
 6   sessions            1000 non-null   int64
 7   total_watch_time    1000 non-null   int64
 8   chat_messages       1000 non-null   int64
 9   bits_cheered        1000 non-null   int64
dtypes: int64(5), str(5)
memory usage: 78.3 KB
```

Compute basic correlations just to see how the data affects eachother in easy to read raw format.

In [55]:
```
corr_matrix = features_df[correlation_cols].corr()

print(corr_matrix.round(2))
```

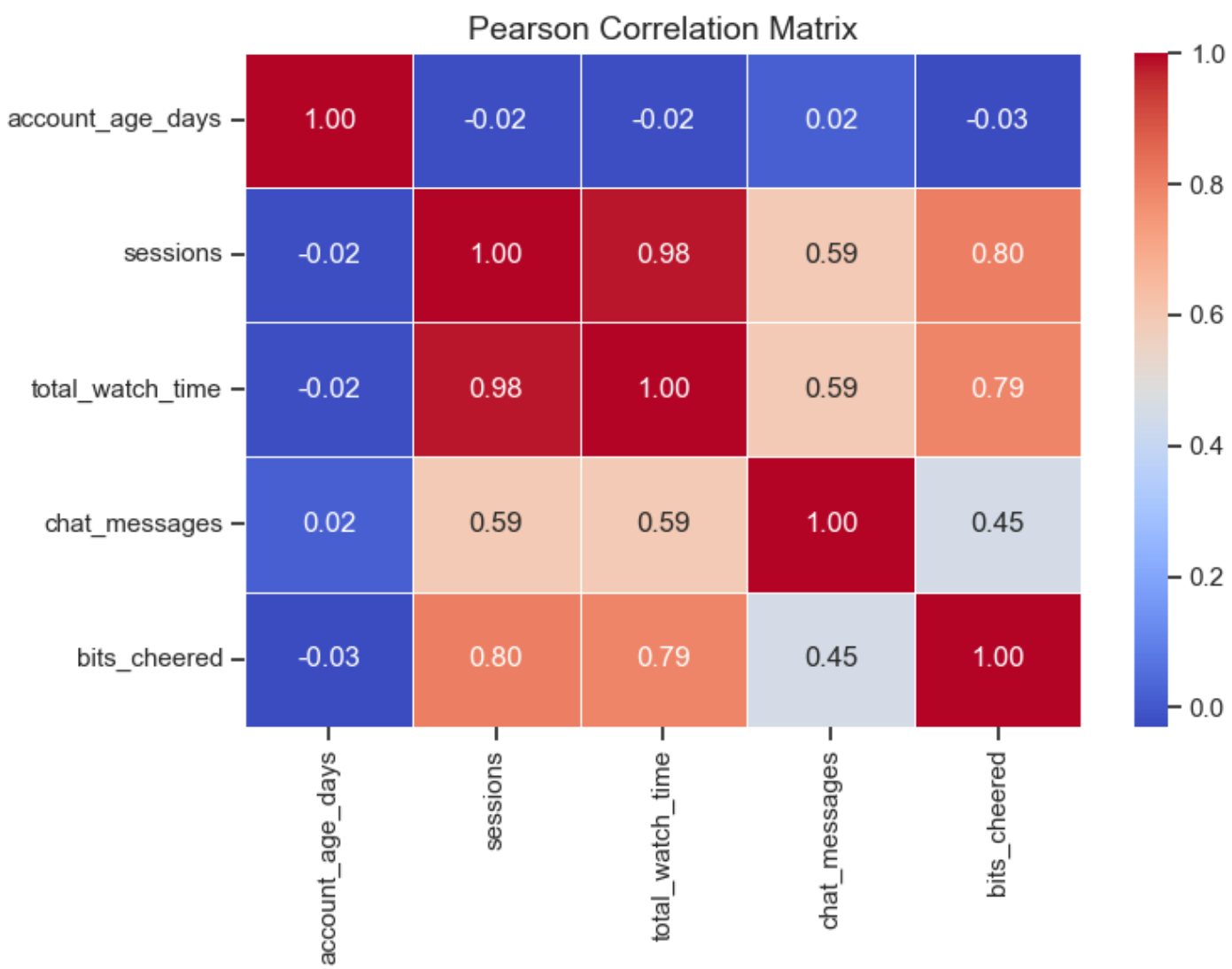
```
                  account_age_days  sessions  total_watch_time  chat_messages  \
account_age_days              1.00     -0.02             -0.02           0.02
sessions                     -0.02      1.00              0.98           0.59
total_watch_time             -0.02      0.98              1.00           0.59
chat_messages                 0.02      0.59              0.59           1.00
bits_cheered                 -0.03      0.80              0.79           0.45

                  bits_cheered
account_age_days         -0.03
sessions                  0.80
total_watch_time          0.79
chat_messages             0.45
bits_cheered              1.00
```

## 2. Visualize the results using a correlation matrix (use a coolwarm color scheme and proper scaling)

### Pearson Correlation Matrix

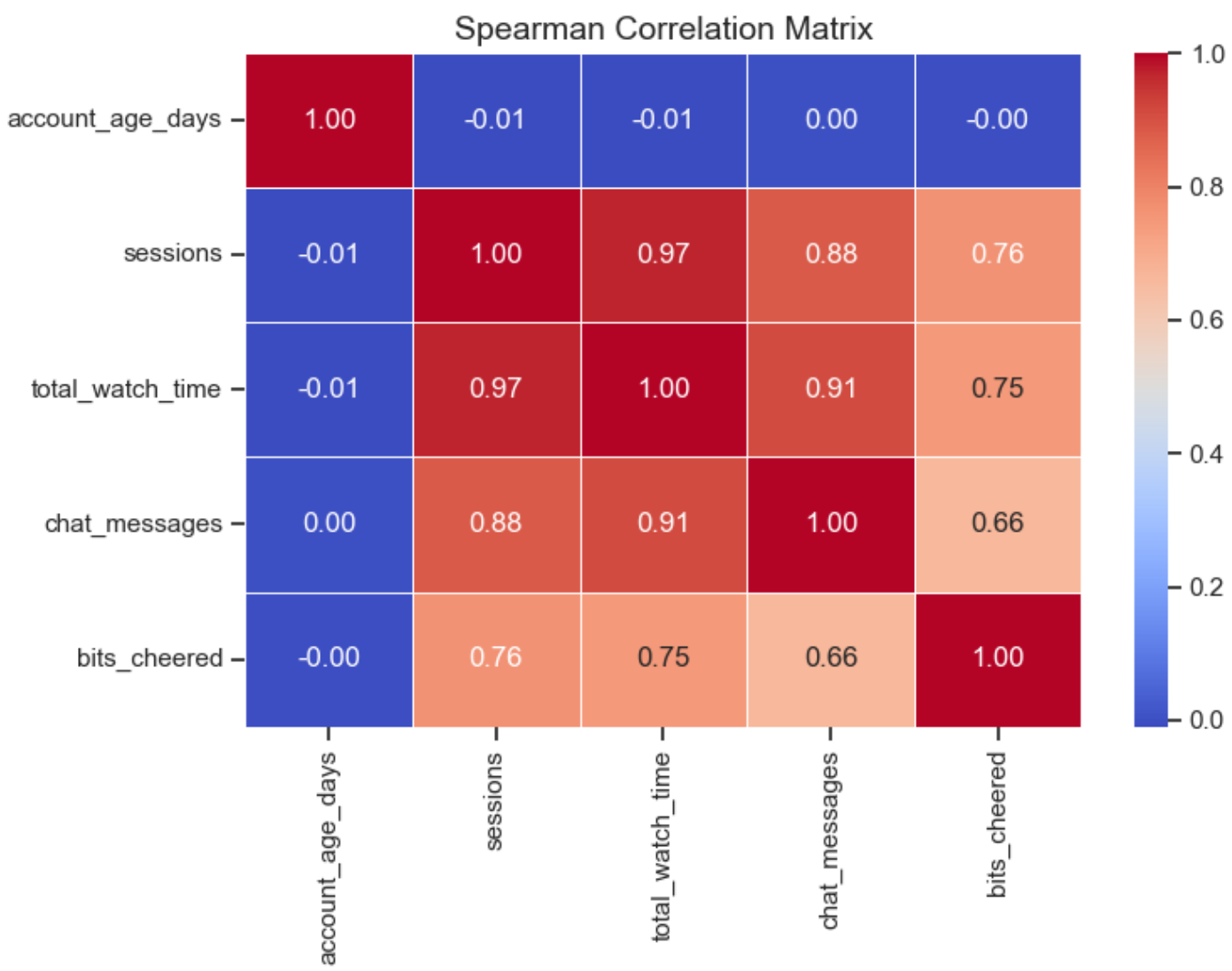
I only realised the default correlation matrix is Pearson after I finished this code, hence why I made `pearson_corr` instead of using the correlation data directly. That means the raw text matrix from before is also the Pearson method and a "default" correlation pandas method doesn't actually exist (its just Pearson)

In [56]:
```
#pearson corelation method
pearson_corr = correlation_data.corr(method="pearson")

plt.figure(figsize=(8, 6))
sns.heatmap(pearson_corr, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Pearson Correlation Matrix", fontsize=14)
plt.tight_layout()
plt.show()
```




## Spearman Correlation Matrix

In [57]:
```
spearman_corr = correlation_data.corr(method="spearman")

plt.figure(figsize=(8, 6))
sns.heatmap(spearman_corr, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Spearman Correlation Matrix", fontsize=14)
plt.tight_layout()
plt.show()
```




## 3. Try Spearman rank correlation and explain when it is more appropriate than Pearson

Pearson measures **linear realtionships** better, so if the data goes up in a straight diagonal line (positive correlation or negative correlation only) it is the ideal choice.

Otherwise the Spearman method measures **monotonic fit** and is stronger/less sensitive against outliers. So if the relationship is non-linear, where values go up and down together regardless of correlation pattern, then Spearman is better.

For this dataset, **Spearman is better for the correlation matrix.** If some extreme fans cheer an extreme amount of bits, Spearman will rank (ex. this is #1 highest bits) it instead of comparing the stats as its literal number. It also works better for ordinal data, like tiers 1-3.

Pearson would be really sensitive to these extreme numbers as it counts the numbers literally without fully taking in the context of the correlation, it just correlates them.

# Task 5 Summary report

## 1. User Characteristics & Age Demographics

- **Demographics by Country:** display the distribution of viewer age groups across different countries.

- **Subscription Tiers:** Isolated viewer counts by grouping categorical tiers using `sort=` to distinguish the free accounts through paid tiers ( `free` -> `tier1` -> `tier2` -> `tier3` ).
- **Average Watch Time by Age:** shared relational columns using an inner `.merge()` on `viewer_id` and `stream_id` . Grouped categories with `.groupby()` and calculated mean with `.mean()`

### 2. Stream Distribution & Overall Platform Metrics

- **Stream Category Sizing:** using `.groupby()` on categories combined with `.nunique()` to count unique instances of `stream_id`
- **Preferred Category Engagement:** Got total viewer engagement by calculating `.sum()` of total watch time per viewer. Merged them back before mapping the totals to a horizontal bar chart.
- **Chronological Trends:** Used `.dt.strftime('%B')` to isolate month names. I divided overall minutes by 60 to convert them into a hours scale using `.agg()`

# Task 6: Compare Behavior Across User Segments

### Monthly watchtime by age-group

In [58]:
```
full_stream_activity.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 22679 entries, 0 to 22678
Data columns (total 24 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   session_id           22679 non-null  str
 1   viewer_id            22679 non-null  str
 2   streamer_id_x        22679 non-null  str
 3   stream_id            22679 non-null  str
 4   started_at_x         22679 non-null  datetime64[us]
 5   ended_at_x           22679 non-null  datetime64[us]
 6   duration_mins        22679 non-null  int64
 7   chat_messages_sent   22679 non-null  int64
 8   bits_cheered         22679 non-null  int64
 9   followed_during      22679 non-null  bool
 10  subscribed_during    22679 non-null  bool
 11  age_group            22679 non-null  str
 12  country              22679 non-null  str
 13  account_age_days     22679 non-null  int64
 14  subscription_tier    22679 non-null  str
 15  preferred_category   22679 non-null  str
 16  streamer_id_y        22679 non-null  str
 17  started_at_y         22679 non-null  datetime64[us]
 18  ended_at_y           22679 non-null  datetime64[us]
 19  stream_duration_hrs  22679 non-null  float64
 20  category             22679 non-null  str
 21  peak_viewers         22679 non-null  float64
 22  title_has_hype_word  22679 non-null  bool
 23  was_raid             22679 non-null  bool
dtypes: bool(4), datetime64[us](4), float64(2), int64(4), str(10)
memory usage: 3.5 MB
```

Article that helped me understand how to properly group data together:

- https://www.geeksforgeeks.org/pandas/python-pandas-dataframe-groupby/

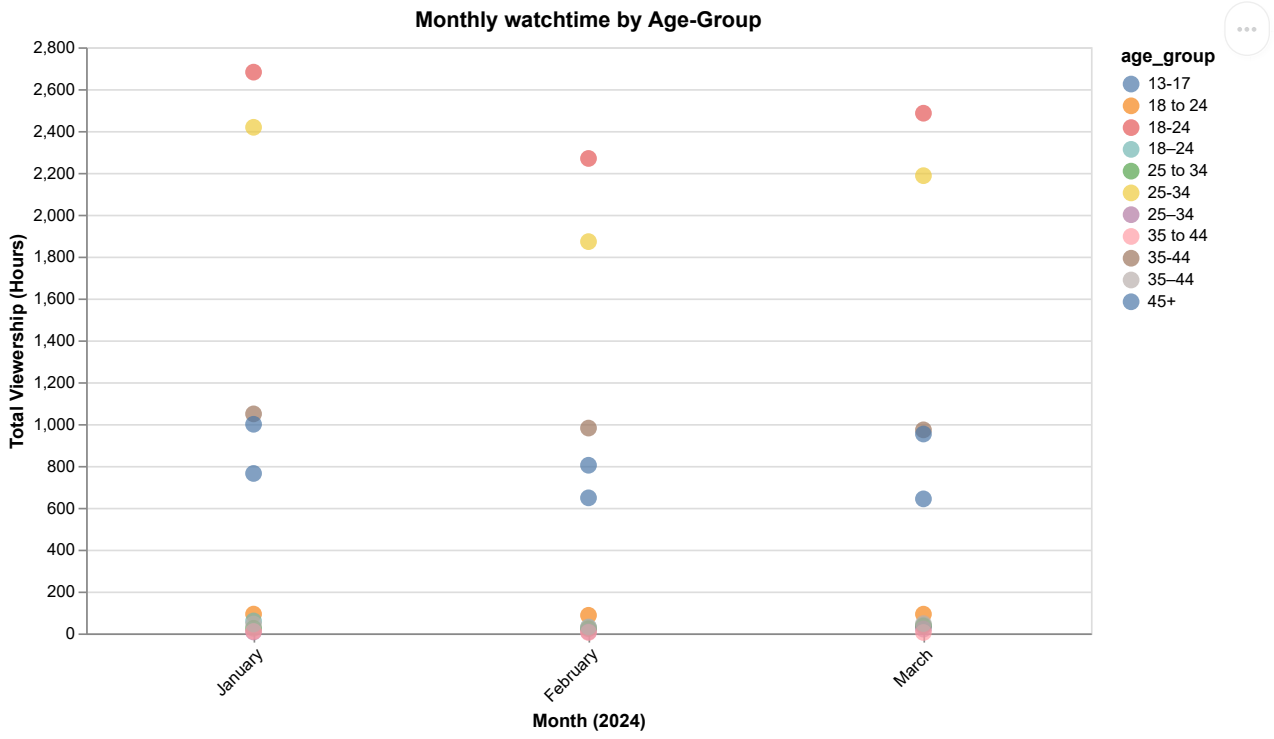
In [59]:
```
#Merge sessions_df with viewers_df to get full behaviour stats
viewer_sessions_df = pd.merge(viewers_df_resolved, sessions_df_resolved, on="viewer_id", how="inn

# Group by age group & month then count individual sessions + add watchtime
aggregated_viewer_sessions_month = viewer_sessions_df.groupby(['month', 'age_group']).agg(
    total_sessions=("session_id","count",),  # distinct watch sessions
    total_watch_hours=("duration_mins", lambda x: x.sum() / 60,),  # Summing total watch hours
).reset_index()

print(aggregated_viewer_sessions_month)
```

| | month | age_group | total_sessions | total_watch_hours |
|---|---|---|---|---|
| 0 | February | 13-17 | 615 | 647.350000 |
| 1 | February | 18 to 24 | 93 | 86.266667 |
| 2 | February | 18-24 | 2384 | 2268.583333 |
| 3 | February | 18–24 | 42 | 28.866667 |
| 4 | February | 25 to 34 | 22 | 22.116667 |
| 5 | February | 25-34 | 1996 | 1871.316667 |
| 6 | February | 25–34 | 10 | 6.883333 |
| 7 | February | 35 to 44 | 4 | 4.266667 |
| 8 | February | 35-44 | 935 | 980.633333 |
| 9 | February | 35–44 | 34 | 30.816667 |
| 10 | February | 45+ | 827 | 803.000000 |
| 11 | January | 13-17 | 760 | 763.883333 |
| 12 | January | 18 to 24 | 93 | 91.983333 |
| 13 | January | 18-24 | 2787 | 2680.900000 |
| 14 | January | 18–24 | 51 | 59.266667 |
| 15 | January | 25 to 34 | 30 | 24.266667 |
| 16 | January | 25-34 | 2454 | 2417.416667 |
| 17 | January | 25–34 | 15 | 6.700000 |
| 18 | January | 35 to 44 | 10 | 9.266667 |
| 19 | January | 35-44 | 1077 | 1048.450000 |
| 20 | January | 35–44 | 55 | 44.683333 |
| 21 | January | 45+ | 1036 | 998.800000 |
| 22 | March | 13-17 | 716 | 642.450000 |
| 23 | March | 18 to 24 | 80 | 91.716667 |
| 24 | March | 18-24 | 2376 | 2484.700000 |
| 25 | March | 18–24 | 38 | 39.333333 |
| 26 | March | 25 to 34 | 27 | 35.166667 |
| 27 | March | 25-34 | 2140 | 2186.366667 |
| 28 | March | 25–34 | 19 | 20.383333 |
| 29 | March | 35 to 44 | 3 | 4.150000 |
| 30 | March | 35-44 | 969 | 972.133333 |
| 31 | March | 35–44 | 44 | 46.783333 |
| 32 | March | 45+ | 937 | 952.300000 |

In [60]:
```
month_order = [
    "January",
    "February",
    "March",
    "April",
    "May",
    "June",
    "July",
    "August",
    "September",
    "October",
    "November",
    "December",
```

```
]

alt.Chart(aggregated_viewer_sessions_month).mark_circle(size=100).encode(
    x=alt.X(
        "month:N",
        title="Month (2024)",
        sort=month_order,
        axis=alt.Axis(labelAngle=-45),
    ),
    y=alt.Y(
        "total_watch_hours:Q",
        title="Total Viewership (Hours)",
        axis=alt.Axis(format=",.0f"),
    ),
    color='age_group:N',
    tooltip=[
        alt.Tooltip("age_group:N", title="Age Group"),
        alt.Tooltip("month:N", title="Month"),
        alt.Tooltip("total_watch_hours:Q", title="Total Watch Hours", format=",.1f"),
        alt.Tooltip("total_sessions:Q", title="Total Sessions", format=","),
        ],
).properties(
    title="Monthly watchtime by Age-Group",
    width=600,
    height=350)
```

Out[60]:




## Monthly Watch Time Between Free & Tier Accounts

In [61]:

```
# Seperate subscription tier into sub group of paid / free
viewer_sessions_df["sub_group"] = viewer_sessions_df["subscription_tier"].apply(lambda x: "Free"

# aggregate session duration sum by month, user group, and unique viewer
sub_monthly_watch = viewer_sessions_df.groupby(["month", "sub_group", "viewer_id"])["duration_mi

# turn minutes into hours
sub_monthly_watch["watch_hours"] = sub_monthly_watch["duration_mins"] / 60

sub_monthly_watch
```

Out[61]:

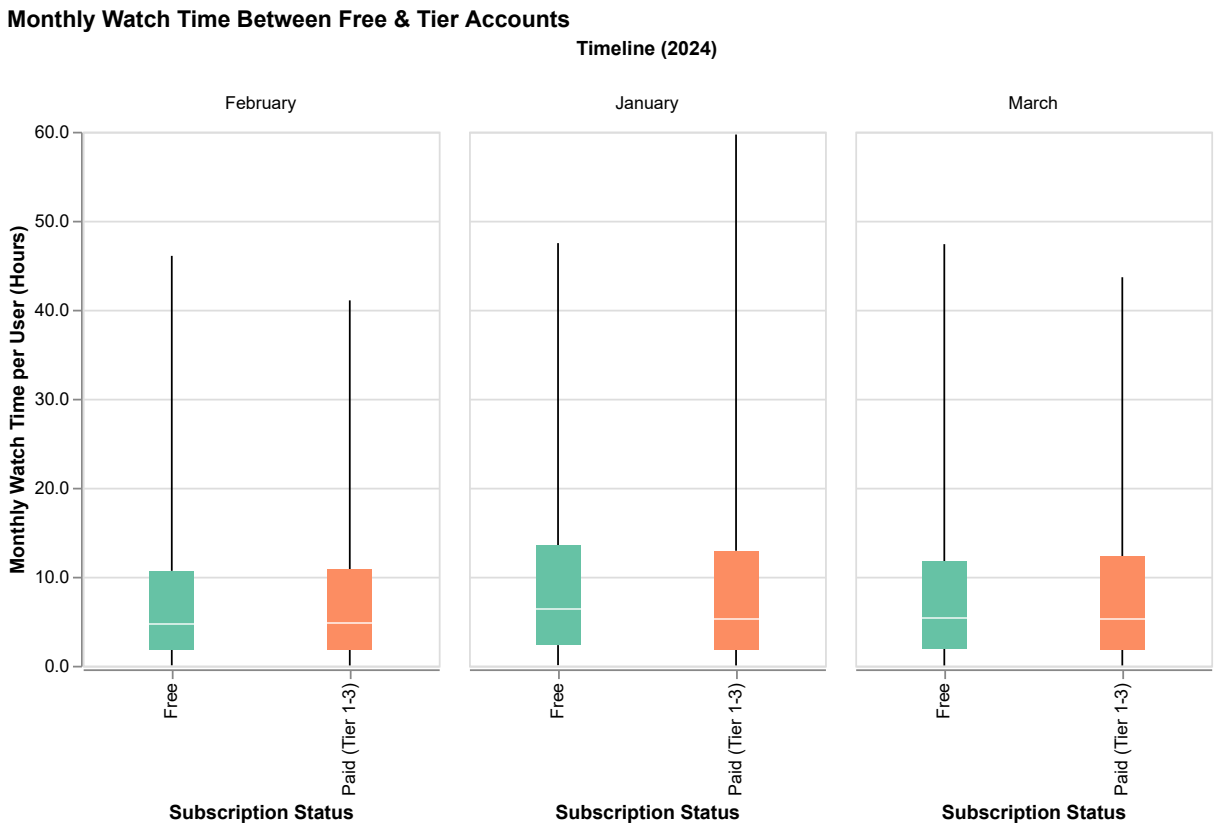
| | month | sub_group | viewer_id | duration_mins | watch_hours |
|---|---|---|---|---|---|
| **0** | February | Free | VWR00002 | 44 | 0.733333 |
| **1** | February | Free | VWR00003 | 272 | 4.533333 |
| **2** | February | Free | VWR00005 | 62 | 1.033333 |
| **3** | February | Free | VWR00010 | 17 | 0.283333 |
| **4** | February | Free | VWR00013 | 117 | 1.950000 |
| **...** | ... | ... | ... | ... | ... |
| **2579** | March | Paid (Tier 1-3) | VWR00989 | 320 | 5.333333 |
| **2580** | March | Paid (Tier 1-3) | VWR00991 | 1137 | 18.950000 |
| **2581** | March | Paid (Tier 1-3) | VWR00992 | 1917 | 31.950000 |
| **2582** | March | Paid (Tier 1-3) | VWR00996 | 1672 | 27.866667 |
| **2583** | March | Paid (Tier 1-3) | VWR00998 | 2379 | 39.650000 |

2584 rows × 5 columns

In [62]:

```
alt.Chart(sub_monthly_watch).mark_boxplot(extent="min-max", size=25).encode(
        x=alt.X("sub_group:N", title="Subscription Status"),
        y=alt.Y(
            "watch_hours:Q",
            title="Monthly Watch Time per User (Hours)",
            axis=alt.Axis(format=",.1f"),
        ),
        color=alt.Color(
            "sub_group:N",
            scale=alt.Scale(domain=["Free", "Paid (Tier 1-3)"], scheme="set2"),
            legend=None,
        ),
        column=alt.Column(
            "month:N", title="Timeline (2024)", sort=month_order, spacing=15
        ),
    ).properties(
        title="Monthly Watch Time Between Free & Tier Accounts",
        width=200,
        height=300,
    )
```

Out[62]:




## Identify any outliers in group behavior and investigate them

I decided to analyze any outlier behaviour between the watch session duration of free users vs. paid ones.

These ended up being anyone who watched a stream for longer than >27 hrs.

In [63]:

```
# run the std outliers function on the dataset
sub_monthly_outliers = detect_outliers(sub_monthly_watch, "watch_hours")
sub_monthly_outliers.info()
print()
print(sub_monthly_outliers['watch_hours'])
print()

#check how much of the original dataset is affected
print(f"Original dataset size: {len(sub_monthly_watch)}")
print(f"Outliers size: {len(sub_monthly_outliers)}")
```

```
<class 'pandas.DataFrame'>
Index: 155 entries, 420 to 2583
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   month          155 non-null    str
 1   sub_group      155 non-null    str
 2   viewer_id      155 non-null    str
 3   duration_mins  155 non-null    int64
 4   watch_hours    155 non-null    float64
dtypes: float64(1), int64(1), str(3)
memory usage: 7.3 KB

420     34.850000
426     32.216667
427     37.983333
428     27.150000
440     31.083333
          ...
2571    28.866667
2576    42.583333
2581    31.950000
2582    27.866667
2583    39.650000
Name: watch_hours, Length: 155, dtype: float64

Original dataset size: 2584
Outliers size: 155
```

In [64]:
```
# check the minimum hours for the outliers
print()
print(sub_monthly_outliers.sort_values(by="watch_hours", ascending=True).head(10))
```

```
         month        sub_group viewer_id  duration_mins  watch_hours
1297   January             Free  VWR00889           1625    27.083333
2176     March             Free  VWR00896           1625    27.083333
2191     March             Free  VWR00923           1626    27.100000
428   February             Free  VWR00865           1629    27.150000
1694   January  Paid (Tier 1-3)  VWR00937           1631    27.183333
1667   January  Paid (Tier 1-3)  VWR00866           1643    27.383333
2216     March             Free  VWR00967           1649    27.483333
1707   January  Paid (Tier 1-3)  VWR00964           1657    27.616667
2217     March             Free  VWR00968           1660    27.666667
2167     March             Free  VWR00881           1662    27.700000
```

A majority of the outliers seem to be free accounts. This means we could potentially investigate why they choose to stay a free account despite their high watchtime.

## Task 6 Summary report

### Age-Based User Behaviour & Comparison

- **User Group Merging:** Combined user profiles and session tracking logs using an inner `.merge()` on the shared tracking ID column as well as `.agg()` to calculate combined watch time.
- **Timeline Visualization:** Plotted the aggregated on a dot plot ( `mark_circle` ), mapping cumulative viewing hours based on both `age_group` and `subscription_tier` (subgrouped into free + paid). Young adults aged 18–24 consistently generated the highest total viewership volumes across the tracked months.

### Behavioral Outliers

- **Subgroup Tiers:** Separated users into two clear consumer groups, "Free" accounts and "Paid (Tier 1-3)" subscribers.
- **Distribution Mapping:** Plotted these distributions as sequential boxplots using `mark_boxplot` to evaluate visual spread over 3 months.
- **Outlier Investigation:** Ran a standard deviation outlier detection helper function on the user metrics, isolating 155 entries. Surprisingly, a major portion of these super-consumers were using entirely free accounts meaning they can benefit from targeted subscription upgrade promotions.

# Task 7: Your Own EDA Questions

## Hypthosis 1: Does title hype-word usage correlate to higher peak viewer count?

Group `streams_df_resolved` on whether the title has a hype word or not and then aggregate the peak viewers using built in math functions.

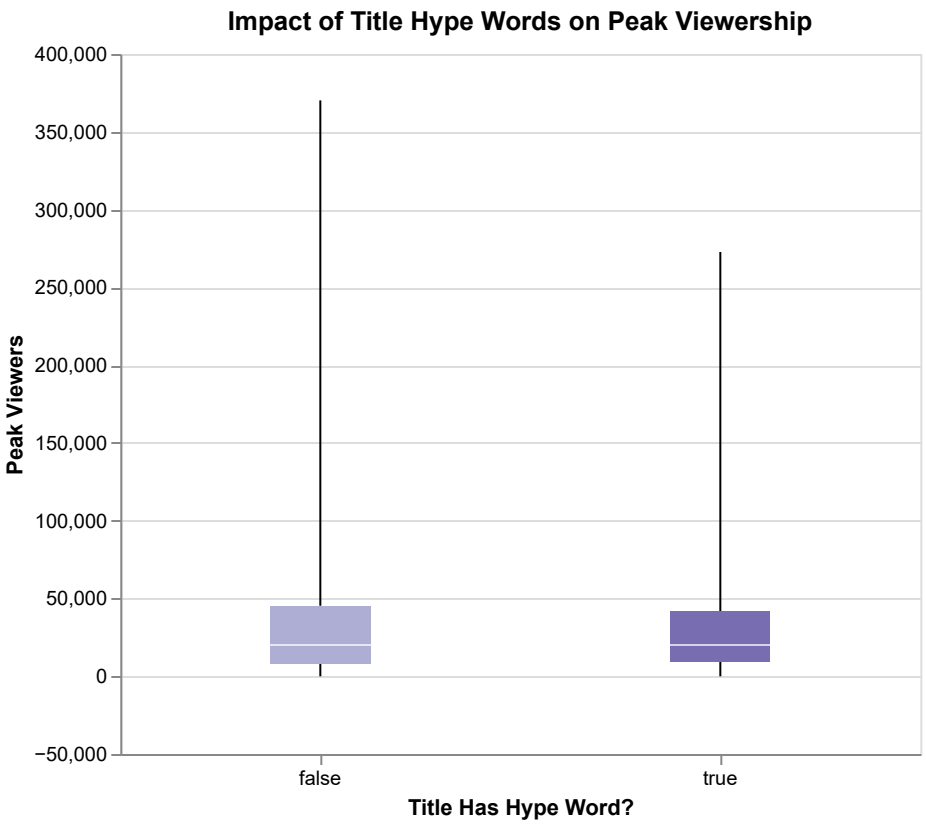
In [65]:
```
# group whether title has a hype word or not and then aggregate the peak viewers using built in m
hype_title_summary = (streams_df_resolved.groupby("title_has_hype_word")["peak_viewers"]
    .agg(["count", "mean", "median", "max"]).reset_index())

hype_title_summary
```

Out[65]:

| | title_has_hype_word | count | mean | median | max |
|---|---|---|---|---|---|
| **0** | False | 853 | 37680.069168 | 20224.0 | 370165.0 |
| **1** | True | 346 | 35893.592486 | 20370.0 | 272708.0 |

In [66]:
```
alt.Chart(streams_df_resolved).mark_boxplot(
    extent="min-max", size=50
).encode(
    x=alt.X(
        "title_has_hype_word:N",
        title="Title Has Hype Word?",
        axis=alt.Axis(labelAngle=0),
    ),
    y=alt.Y(
        "peak_viewers:Q",
        title="Peak Viewers",
        axis=alt.Axis(format=",.0f"),
    ),
    color=alt.Color(
        "title_has_hype_word:N",
        scale=alt.Scale(scheme="purples"),
        legend=None,
    ),
).properties(
    title="Impact of Title Hype Words on Peak Viewership", width=400, height=350)
```

Out[66]:




**Conclusion**

Suprisingly, clickbait "hype word" in the title does not drive extra traffic. The max viewership of streams without the words, `false`, actually exceeds those that are `true`. The average viewership is slightly higher as well. Therefore hypewords don't correlate with getting more viewers and seems to have the opposite effect.

- Though the Q1 and the median are higher when `true` the Q3 isn't. This might suggest outliers are skewing the data.

## Hypthosis 2: Are morning or evening streams more popular with viewers and what category do they belong to?

First I have to do the same thing I did last time when I extracted the month but with the hour of the day this time. Since we use a 24 hour timeframe, we can just segment the pandas datetime object into different sections of the day.

- After that we group them by category and aggregate the viewership to them

In [67]:

```
# create new stream hours column
streams_df_resolved["stream_hour"] = streams_df_resolved["started_at"].dt.hour

# function to automatically sub-group them without doing it manually
# since it's 24 hour time it's easy to seperate them into groups
# Morning = 5AM-12PM
# Evening = 5-11PM

def assign_time_of_day(hour):
    if 5 <= hour < 12:
        return "Morning"
    elif 17 <= hour <= 23:
        return "Evening"
```

```
    else:
        return "Other"

# use the function to assign time of day to every stream in new sub group column
streams_df_resolved["time_of_day"] = streams_df_resolved["stream_hour"].apply(assign_time_of_day)

# Group time of day and category together, then aggregate peak viewers to them
time_category_summary = (streams_df_resolved.groupby(["time_of_day", "category"])["peak_viewers"]
    .agg(["count", "mean", "median"])
    .reset_index())

# #Preview
time_category_summary.head(2)
```

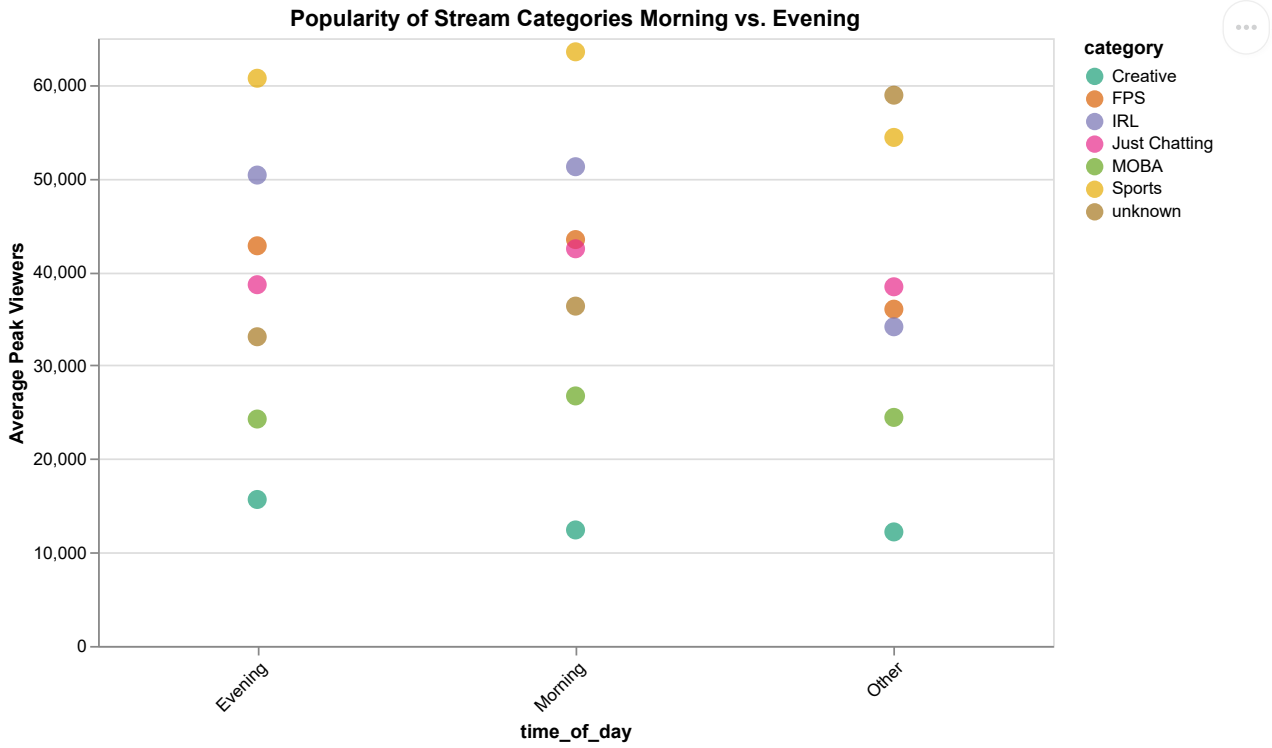
Out[67]:

| | time_of_day | category | count | mean | median |
|---|---|---|---|---|---|
| **0** | Evening | Creative | 76 | 15663.276316 | 13484.0 |
| **1** | Evening | FPS | 103 | 42797.097087 | 32405.0 |

In [68]:

```
alt.Chart(time_category_summary).mark_circle(size=120).encode(
    x=alt.X(
        "time_of_day:N",
        axis=alt.Axis(labelAngle=-45),
    ),
    y=alt.Y(
        "mean:Q",
        title="Average Peak Viewers",
        axis=alt.Axis(format=",.0f"),
    ),
    color=alt.Color(
        "category:N",
        scale=alt.Scale(scheme="dark2"),
    ),
    tooltip=[
        alt.Tooltip("time_of_day:N", title="Time"),
        alt.Tooltip("category:N", title="Category"),
        alt.Tooltip("mean:Q", title="Avg Peak Viewers", format=",.0f"),
        alt.Tooltip("count:Q", title="Total Streams"),
    ],
).properties(
    title="Popularity of Stream Categories Morning vs. Evening",
    width=550,
    height=350)
```

Out[68]:




**Conclusion**

Morning streams preform better overrall for almost every category except `Creative`. `Just Chatting` reaches the highest peak viewership of any category during the morning. `IRL` viewership drops significantly past 11pm.

- Morning viewership seems to be the most popular and common.
- Meanwhile variety `unknown` streams surpass `Just Chatting` after 11pm.

## Briefly discuss what additional data would help you answer the questions more effectively

**To better evaluate Hype Titles:** Right now, we only see the final peak viewership, but we don't know how many users saw the title in their dashboard feed and explicitly chose to ignore it vs. click on it. Maybe some sort of algorithim data would help us understand how many viewers were exposed to these streams.

**To better evaluate Stream Timing:** Because Twitch is a global platform, a stream that begins in the morning for a European creator is broadcasting live to midnight in America. Matching the viewers timezone to the Streamer's timezone would add a lot of context

- Though I don't know how this could be graphed or visualized.

## Task 7 Summary report

### 1. Hypothesis 1: Title Hype Words vs. Peak Viewers

- **Data Processing:** Grouped the broadcast data based on whether a stream title included a clickbait hype word or not using `.groupby()`. Calculated the total count, mean, median, and maximum peak viewers using `.agg()`.
- **Findings:** Surprisingly, using hype words in the title does not bring in more viewers. Streams without

hype words actually had a higher maximum viewership and a slightly higher average viewer count. While streams with hype words had a slightly higher median, the upper boundaries suggest that standard outliers might be twisting the raw metrics.

### 2. Hypothesis 2: Stream Timing and Category Popularity

- **Data Processing:** Extracted the raw hour from the stream start times using `.dt.hour`. Built a custom Python function to automatically sort these hours into "Morning", "Evening", or "Other" buckets, and applied it to the dataset using `.apply()`. Grouped the data by both time of day and stream genre to find the average viewer counts.

- **Findings:** The chart showed that morning streams actually do better overall for almost every single category except for Creative streams. `Just Chatting` brings in the biggest morning crowds, `IRL` drops off heavily late at night, and unclassified variety streams take over the top spot after 11 PM.

### 3. Future Data Improvements

- **Hype Word Tracking:** To really understand if clickbait titles work, it would help to have algorithim data. We only see the final viewer peaks but we do not know how many users were exposed to the stream.

- **Time Zone Adjustments:** Having the physical location or time zone of the viewers would make the stream timing data much more accurate.

# Task 8: EDA Presentation (Insight Report)

Summarize your findings in a polished format:

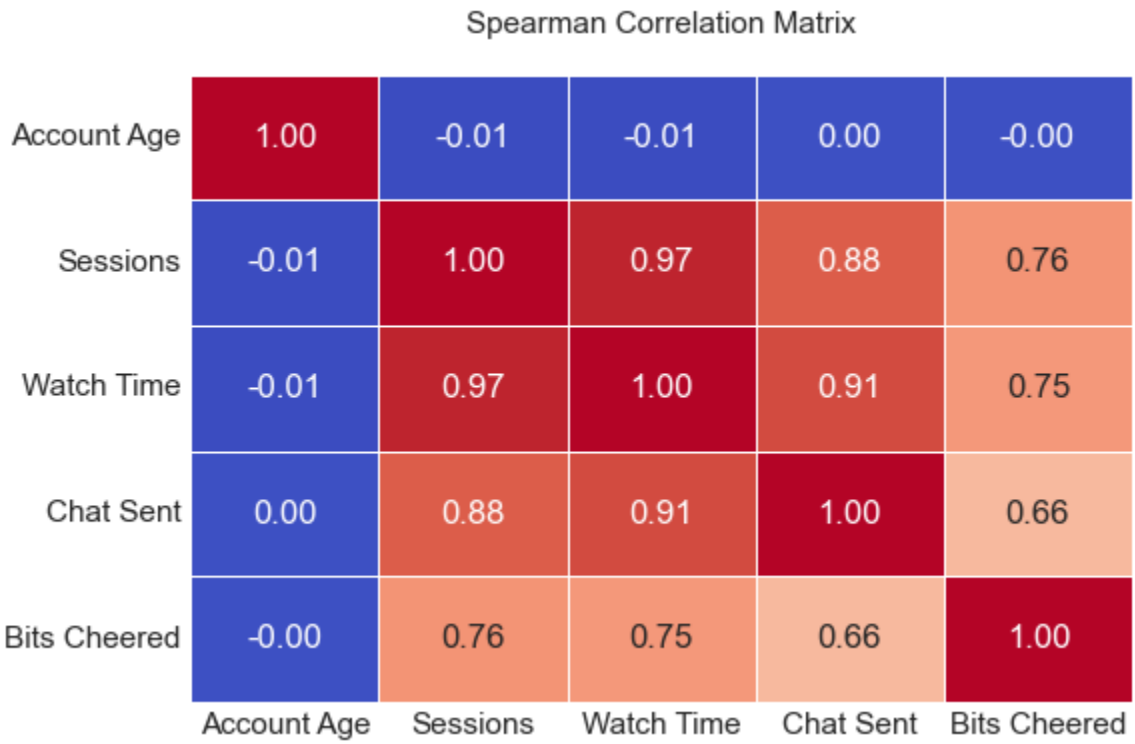
- Create 3–4 slides or a single-page report highlighting your most important insights
- Include at least 2 charts and interpret them using clear, plain language
- Apply Tufte's principles: minimize chartjunk, maximize data-to-ink ratio, and ensure clear labeling

**Please see .pdf for the single-page report**

## Edward Tuft Design Principles

In [69]:
```
plt.figure(figsize=(6, 4))

# convert the dataframe into matrix format for easy manipulation/display
# and define labels
labels = ["Account Age", "Sessions", "Watch Time", "Chat Sent", "Bits Cheered"]
matrix_data = np.array(
    [
        [1.00, -0.01, -0.01, 0.00, -0.00],
        [-0.01, 1.00, 0.97, 0.88, 0.76],
        [-0.01, 0.97, 1.00, 0.91, 0.75],
        [0.00, 0.88, 0.91, 1.00, 0.66],
        [-0.00, 0.76, 0.75, 0.66, 1.00],
    ]
)

# Plotting with a clean, low-ink framework
ax = sns.heatmap(
```

```
    matrix_data,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    xticklabels=labels,
    yticklabels=labels,
    cbar=False,  # eliminate the legend/color bar
    linewidths=0.5,
)

plt.title("Spearman Correlation Matrix", fontsize=11, pad=15)
plt.tick_params(axis="both", which="both", length=0)  # Removes tick mark
plt.tight_layout()
plt.show()
```




In [70]:

```
# isolate the two categories we want to focus on 'Just Chatting' & 'FPS'
chart_data = pd.DataFrame(
    {
        "time_of_day": ["Evening", "Evening", "Morning", "Morning"],
        "category": ["Creative", "FPS", "Creative", "FPS"],
        "avg_peak_viewers": [15663, 42797, 28500, 56000],
    }
)

# high-contrast dot plot
alt.Chart(chart_data).mark_circle(size=140).encode(
    x=alt.X(
        "time_of_day:N", title="Broadcast Window", axis=alt.Axis(labelAngle=0)
    ),
    y=alt.Y(
        "avg_peak_viewers:Q",
        title="Average Peak Viewership Count",
        axis=alt.Axis(format=",.0f", tickCount=5),
    ),
    color=alt.Color(
        "category:N",
        title="Content Category",
        scale=alt.Scale(scheme="dark2"),
```

```
    ),
).properties(
    title="Average Peak Audience Sizes: Morning vs. Evening",
    width=350,
    height=350,
).display()
```